CUDA 编程优化

面试 100 题

从 CUDA 执行模型到 Hopper / Blackwell Kernel Optimization

参考《剑指 Offer》的“题目驱动 + 层层追问”学习方式

原创内容 · CUDA 13.3 口径 · 2026-09

适用岗位 CUDA / GPU Kernel Engineer · HPC · AI Infra · LLM Inference · GPU Compiler · NVIDIA DevTech 风格面试

核心方法：Memory Efficiency × Compute Efficiency × Parallelism × Latency Hiding × Resource Balance

前言：这不是一本“背答案”的 CUDA 题库

CUDA 面试最常见的误区，是把大量术语背成孤立结论：shared memory 很快、occupancy 越高越好、atomic 很慢、256 threads 最优。真正的高阶面试会继续追问：为什么？在什么条件下？用什么 profiler 指标证明？如果换到 Hopper / Blackwell 还成立吗？

本书以 100 道题为骨架，每道题都按照“考察意图 → 思路拆解 → 标准回答 → Gotchas → 追问”展开。对 Reduction、Scan、Histogram、Transpose、GEMM 等 coding 题给出可讨论的 CUDA C++ 骨架；对 Nsight、Roofline、TMA、warp specialization、TMEM、CUDA Tile 等题强调性能因果链。

“真题”口径分两类：少数题可在公开面经中直接核验，例如 synchronize、reduction、memory hierarchy，以及 software pipelining、warp specialization；其余题按 CUDA/GPU 岗长期高频考点构造成真题型问题。不会把无法核验的题虚构成某公司逐字原题。

本书参考《剑指 Offer》的学习方法，而不复制其内容、文字或版式：先给问题，再给分析路径，最后给可以在面试中直接组织成语言的答案。

> **30 秒回答法** 先给一句结论，再给 3 个关键因果点；如果面试官继续追问，再进入硬件交易、资源约束、代码和 profiler 证据。

当前技术口径

- CUDA Toolkit 13.3 官方文档作为基础版本口径；CUDA 13.3 已引入 CUDA Tile C++。
- 经典优化遵循 CUDA Best Practices：coalescing、shared-memory reuse、bank conflict、occupancy、register/spill、asynchronous copy。
- Hopper 重点：Thread Block Cluster、Distributed Shared Memory、TMA、异步 barrier 与 warp-specialized pipelines。
- Blackwell 重点：TCGen05、Tensor Memory（TMEM）、新的 CUTLASS/CuTe data locale 与不同于 Hopper 的 warp specialization 结构。
- 性能诊断重点：Nsight Systems → Nsight Compute → Roofline/SOL → stalls/resource/traffic 的因果闭环。

学习地图：四轮掌握 100 题

轮次	题号	目标	通过标准
第一轮	1-40	执行模型 + Memory	能在 30-60 秒内准确回答核心概念，并能手算 coalescing/bank mapping。
第二轮	41-60	资源模型 + Coding	能现场解释 occupancy/register trade-off，并写 Reduction/Transpose/Scan/GEMM。
第三轮	61-80	Async + Profiling	能用 pipeline、Roofline、Nsight 从指标定位瓶颈。
第四轮	81-100	现代高性能 CUDA	能解释 CUTLASS、Tensor Core、TMA、warp specialization、TMEM、CUDA Tile。

十章目录

第一章 CUDA 执行模型与 SIMT 从“线程”进入“warp 调度”的硬件视角。

第二章 GPU Memory Hierarchy 理解性能优化的真正成本中心：数据在哪里、如何移动。

第三章 Global Memory 与 Coalescing 把 warp 访问模式翻译成 memory transactions。

第四章 Shared Memory、Warp 与 Divergence 片上协作、bank conflict 与 warp-level 编程。

第五章 Occupancy、Register 与 Latency Hiding 资源约束、TLP/ILP 与“高 occupancy 迷思”。

第六章 经典 CUDA Kernel 现场编程 Reduction、Scan、Histogram、Transpose、GEMM 等必练题。

第七章 Stream、异步拷贝与 Software Pipeline 从“能并行”升级到“让搬运与计算持续重叠”。

第八章 Nsight、Roofline 与性能诊断 建立可证伪、可复现的性能工程方法。

第九章 GEMM、Tensor Core 与 CUTLASS 理解现代 AI Kernel 的分层 tiling 与 warp specialization。

第十章 Hopper、Blackwell 与 CUDA 13.x Thread Block Cluster、DSM、TMA、TMEM、CUDA Tile 与工程化优化。

第一章 CUDA 执行模型与 SIMT

从“线程”进入“warp 调度”的硬件视角。

本章抓手 线程层级与硬件调度实体 · warp 视角 · 同步与异步 launch

题 01 | Thread、Warp、Block、Grid、SM 分别是什么？

难度	★☆☆	频率	极高	标签	执行模型 / 基础

30 秒答案 编程层级是 Grid→Block→Thread；硬件调度核心是 SM→Warp，warp=32 threads。

考察意图

看你能否把 CUDA 的编程抽象与 GPU 的硬件调度实体分开。

思路拆解

最容易失分的点，是把 warp 当成程序员显式创建的对象，或者说一个 block 会被拆到多个 SM 上执行。面试回答应该先画出软件层级，再映射到硬件。

标准回答

- Grid 是一次 kernel launch 的全部 thread blocks；Block 是线程协作与共享内存的基本范围；Thread 是 CUDA 暴露的逻辑执行单元。
- 硬件按 warp 调度线程。当前 NVIDIA CUDA 的 warp 为 32 threads；一个 SM 可以同时驻留多个 blocks 和 warps。
- 一个 thread block 在其生命周期内驻留在一个 SM 上，不会跨 SM 拆分；不同 blocks 可以被动态分配到任意可用 SM。
- 性能分析时要从“单线程做什么”升级为“同一 warp 的 32 个线程执行什么指令、访问什么地址”。

Gotchas：常见失分点

- 把 SM 说成一个 CUDA core。
- 认为 block 与 SM 是一一对应关系。

面试官可能继续追问

- 为什么 warp size 会影响 block size 选择?
- 一个 SM 能同时驻留多少 blocks 由哪些资源决定?

参考：R1, R2

题 02 | CUDA 为什么称 SIMT，而不是传统 SIMD？

难度	★☆☆	频率	高	标签	SIMT / 架构

考察意图

考察你是否理解 CUDA “线程语义”与底层矢量化执行之间的关系。

思路拆解

SIMD 把 lane 暴露给程序员，而 SIMT 给每个 CUDA thread 独立的线程语义、寄存器状态和控制流，再由硬件把线程聚成 warp。这个差异直接解释了 divergence 为什么存在。

标准回答

- SIMD 的抽象是“一条显式向量指令操作多个 lane”；SIMT 的抽象是“多个逻辑线程被硬件以 warp 形式共同发射”。
- 每个 CUDA thread 有自己的 threadIdx、寄存器和逻辑控制流状态；但 warp 中 active lanes 通常共享同一条被发射的指令。
- 发生分支时，SIMT 能表达不同线程走不同路径；硬件通过 active mask/重汇合机制执行这些路径，因此可能降低 lane 利用率。

Gotchas：常见失分点

- 把 SIMT 解释成“32 个 CUDA core 同时执行”。
- 忽略 independent thread scheduling 对 warp-synchronous 代码的影响。

面试官可能继续追问

- Volta 之后 independent thread scheduling 改变了什么?
- 为什么 warp-level primitive 需要 mask?

参考：R1, R8

题 03 | 为什么 Warp 是 CUDA 性能分析最重要的单位？

难度	★☆☆	频率	极高	标签	Warp / 性能

考察意图

考察你能否从硬件的 transaction、scheduler、branch 行为解释性能。

思路拆解

CUDA 的很多“看似线程级”问题最终都以 warp 为粒度体现：global memory transaction、shared-memory bank conflict、branch divergence、warp shuffle、scheduler eligibility。

标准回答

- Global memory coalescing 是把一个 warp 的地址请求合并成尽量少的 memory transactions。

- Bank conflict 发生在同一 warp 的 shared-memory 请求映射到同一 bank 的不同地址时。
- 分支是否伤性能，关键是同一 warp 内的 lanes 是否走不同路径，而不是代码中是否存在 if。
- Nsight Compute 的许多 stall 与 issue 指标也是以 warp/scheduler 视角解释。

Gotchas：常见失分点

- 只看 thread-level instruction count，而不看 warp 行为。

面试官可能继续追问

- 为什么一个跨 warp 的 branch 通常不产生 divergence?
- warp shuffle 为什么可以替代部分 shared-memory 通信?

参考：R2, R4

题 04 | Kernel launch 后，Block 是如何调度到 SM 的？

难度	★★☆	频率	高	标签	调度 / Occupancy

考察意图

考察资源约束和动态 block 调度的理解。

思路拆解

Block 调度不是静态绑定。SM 只有在 register、shared memory、thread/warp slots、block slots 等资源都够时，才可以让新 block resident。

标准回答

- Grid 中 blocks 可被动态分派给可用 SM，程序通常不应依赖某个 block 被放在哪个 SM。
- 一个 SM 的 resident blocks 上限是多个约束的最小值：线程数、warp 数、寄存器、shared memory、block slot 等。
- 当一个 resident block 完成，释放资源后，SM 才能接纳后续 block；因此 grid 足够大才能形成持续的 waves。
- 尾部只剩少量 blocks 时，可能出现 tail effect，即一部分 SM 已空闲但 kernel 尚未结束。

Gotchas：常见失分点

- 把 block 调度顺序当作确定的。
- 依赖 blockIdx 推断物理 SM。

面试官可能继续追问

- 什么是 wave quantization / tail effect?
- persistent kernel 为什么会改变 block 工作方式?

参考：R1, R2

题 05 | 为什么 block size 常从 128/256 开始，而不是随便选？

难度	★☆☆	频率	极高	标签	Block Size / Warp

考察意图

考察 block size 与 warp 填充、资源占用、延迟隐藏之间的权衡。

思路拆解

“256 最快”不是规律。合理起点来自两个事实：threads/block 应是 32 的倍数以减少 under-populated warp；同时 128-256 往往能在并行度与 block 数之间提供不错的初始平衡。最终必须 profile。

标准回答

- blockDim 最好是 warp size 的整数倍；例如 100 threads 会生成 4 个 warps，其中最后一个只有 4 个 active lanes。
- 较大 block 可增加单 block 并行度，但也会增加每 block 的 register/shared-memory 总消耗。
- 较小 block 可能让更多 blocks 同时 resident，有利于隐藏 barrier 或 memory latency。
- 官方 Best Practices 把 128-256 作为实验起点，而非固定答案。

Gotchas：常见失分点

- 回答“256 是官方推荐，所以一定最优”。

面试官可能继续追问

- 64、128、256、512 应该怎么系统 benchmark?
- block 维度形状会影响 coalescing 吗?

参考：R2

题 06 | 什么是 Grid-Stride Loop？为什么它是 CUDA 常用模式？

难度	★☆☆	频率	高	标签	索引 / 工程

考察意图

考察是否具备可扩展 kernel 的基本编码习惯。

思路拆解

Grid-stride loop 让线程处理多个逻辑元素，解耦数据规模 N 与 launch 的 block 数；它还能控制 grid 大小、复用线程上下文，并方便做 persistent-style 工作分配。

标准回答

- 初始索引 i = blockIdx.x * blockDim.x + threadIdx.x。
- 步长 stride = blockDim.x * gridDim.x；每个线程处理 i, i+stride, ...。
- 连续线程的第一次访问仍保持连续，因此可以保留 coalescing。
- 它允许将 gridDim 调到与 SM 数、work chunking 等更适配，而不是盲目发 N/threads 个 blocks。

代码 / 白板骨架

```
__global__ void saxpy(int n, float a, const float* x, float* y) {
    for (int i = blockIdx.x * blockDim.x + threadIdx.x;
         i < n;
         i += blockDim.x * gridDim.x) {
        y[i] = a * x[i] + y[i];
    }
}
```

Gotchas：常见失分点

- stride 写成 blockDim.x 而忽略 gridDim.x。
- 把 grid-stride loop 与 strided memory access 混为一谈：前者每一轮 warp 内仍可连续。

面试官可能继续追问

- 什么时候 grid-stride loop 会降低 locality?
- persistent kernel 与 grid-stride loop 的区别?

参考：R1

题 07 | `__syncthreads()` 到底同步什么？

难度	★★☆	频率	极高	标签	同步 / 内存模型

考察意图

考察 barrier 语义，而不是只会背“同步 block”。

思路拆解

正确答案至少包含两层：它是 block 范围的 barrier；同时对参与线程建立内存顺序，使 barrier 前的共享/全局内存操作在相应语义下对参与线程可见。

标准回答

- 同一个 thread block 中所有尚未退出且需要参与该 barrier 的线程必须到达同一个同步点。
- `__syncthreads()` 常用于“shared memory 生产者写完 → 消费者读取”之间。
- 把 barrier 放在非一致控制流中非常危险：如果部分线程到达、部分线程永远到不了，行为是错误的。
- 同步不是缓存刷新魔法；跨 block 通信需要更高层同步/原子/内存顺序机制。

代码 / 白板骨架

```
// 错误范式：条件可能在 block 内不一致
if (threadIdx.x < valid) {
    __syncthreads();
}

// 更常见的正确思路：所有线程都到 barrier，
// 只有读写本身按 valid 条件屏蔽。
```

Gotchas：常见失分点

- 在 `if (tid < n) __syncthreads();` 中让不同线程条件不同。
- 认为 `__syncthreads()` 可以同步整个 grid。

面试官可能继续追问

- `__threadfence()` 与 `__syncthreads()` 有何区别?
- 什么时候使用 `cuda::barrier`?

参考：R7, R9

题 08 | CUDA 有哪些同步层级？

难度	★★☆	频率	高	标签	同步 / Cooperative Groups

考察意图

考察你能否构建从 warp 到 system 的同步层级。

思路拆解

CUDA 同步成本随作用域扩大而增加。回答时应明确“barrier”和“memory ordering”不是同一个概念，且不同 primitives 有不同 scope。

标准回答

- warp：`__syncwarp(mask)` 和 warp-level primitives。
- block：`__syncthreads()`、block-scoped barrier。
- cluster：Hopper/CC 9.0+ 的 cluster synchronization。
- grid：cooperative launch/grid group 或 kernel boundary；普通 blocks 不能随意做全 grid barrier。
- host-device / stream：events、stream waits、`cudaDeviceSynchronize()` 等。

Gotchas：常见失分点

- 把 atomic 当成全局 barrier。
- 忽略 thread scope 与 memory order。

面试官可能继续追问

- 什么是 release/acquire?
- 为什么跨 GPU/system scope 更昂贵?

参考：R1, R7

题 09 | 为什么 kernel launch 通常是异步的？如何正确计时？

难度	★☆☆	频率	极高	标签	异步 / Benchmark

考察意图

考察是否会做正确 benchmark。

思路拆解

Host 发起 kernel launch 通常只是把工作 enqueue 到 stream；如果直接用 CPU wall-clock 包住 launch 而不等待完成，测到的可能主要是 launch 时间。

标准回答

- 同 stream 内操作保持顺序，但 host 可在 GPU 工作未完成时继续执行。
- 测 kernel 时间优先使用 CUDA events，在同一 stream 记录 start/stop，并同步 stop event。
- 端到端时间要包含用户真正关心的 H2D、kernel、D2H、同步和 CPU orchestration。
- benchmark 要 warm-up、多次迭代并报告统计量。

Gotchas：常见失分点

- 用 `std::chrono` 只包 kernel launch 就宣称 kernel 耗时。

面试官可能继续追问

- CUDA event 计时与 Nsight timeline 有什么差别?
- 为什么频繁 device-wide synchronize 会破坏 overlap?

参考：R1, R10

题 10 | 为什么 GPU 更偏 throughput，而 CPU 更偏 latency？

难度	★☆☆	频率	高	标签	架构 / Latency Hiding

考察意图

考察你是否理解 GPU 为何需要大量线程。

思路拆解

GPU 不追求把单个 DRAM load 的延迟变得极低，而是保存大量 warp 状态，在一个 warp 等待时发射其他可运行 warp，从而用并行度覆盖延迟。

标准回答

- CPU 用复杂缓存、分支预测、乱序执行等机制优化少量线程的单线程 latency。
- GPU 投入更多面积到算术吞吐与大量 resident threads，依赖 warp scheduling 隐藏 latency。
- 因此 GPU 需要足够并行工作量；小任务、强串行依赖、频繁全局同步通常难以发挥 GPU。
- 这也是 occupancy/TLP 与 ILP 成为 CUDA 性能优化核心变量的原因。

Gotchas：常见失分点

- 把 GPU 说成“每个 core 比 CPU core 更快”。

面试官可能继续追问

- 什么情况下 CPU 版本反而可能更快?
- GPU 的 register file 为什么很大?

参考：R2

本章复盘：面试前必须能说清的因果链

- Thread 是编程抽象，warp 是调度与性能分析核心。
- Block 不跨 SM；resident blocks 由资源约束决定。
- 同步必须同时考虑 barrier scope 与 memory ordering。

第二章 GPU Memory Hierarchy

理解性能优化的真正成本中心：数据在哪里、如何移动。

本章抓手 register/shared/cache/HBM · spilling · pinned/unified memory

题 11 | 请完整描述 GPU Memory Hierarchy。

难度	★☆☆	频率	极高	标签	Memory / 基础

30 秒答案 Register/Shared 在片上；Global/Local/Constant/Texture 主要驻留 device memory；L1/L2 是硬件 cache。

考察意图

这是公开 CUDA 面经可核验的高频题，核心是理解不同空间的 scope、latency 与管理方式。

思路拆解

回答不应只背“register/shared/global”，还要指出 local memory 虽然 thread-private，但通常位于 device memory；L1/L2 是 cache 层级，shared 是显式管理 scratchpad。

标准回答

- Registers：on-chip、thread-private、最快但容量受限。
- Shared memory：on-chip、block-scoped、显式管理，可用于 reuse、reordering、协作。
- L1/texture/read-only cache：SM 侧硬件管理缓存路径；L2 是所有 SM 共享的 device-level cache。
- Global memory/HBM：容量大、延迟高、带宽高。Local memory 逻辑上属于线程，但物理上通常落到 device memory 并由 cache 缓解。
- Constant memory 适合 warp 内相同地址的广播式读取。

Gotchas：常见失分点

- 说 local memory 在 SM 上。
- 把 shared memory 与 L1 完全等同。

面试官可能继续追问

- 哪些数据应该放 shared，而不是依赖 L1?
- L2 对跨 block 复用有什么意义?

参考：R2, R1, R11

题 12 | Register 为什么快？为什么又不能无限使用？

难度	★★☆	频率	高	标签	Register / 资源

考察意图

考察 register file 的双重角色：低延迟存储与 occupancy 限制。

思路拆解

Registers 是 thread-private 的片上存储，编译器尽量把标量和小型静态对象留在 register。但 register file 是每 SM 共享的有限资源，per-thread 使用量越高，能同时驻留的 threads/warps/blocks 越少。

标准回答

- register access 延迟低，适合 accumulator、循环变量和 thread tile。
- 总 register 需求大致随 registers/thread × resident threads 增长，并按架构粒度分配。
- 过多 register 会降低 occupancy；过少则可能造成 spill，增加 local-memory traffic。
- 高性能 kernel 往往有意接受较低 occupancy 来换取更大的 register tile 和 ILP。

Gotchas：常见失分点

- “寄存器越少越快”。
- 只看 occupancy，不看 spills。

面试官可能继续追问

- 如何查看 register usage?
- 为什么 GEMM consumer warp 常常很吃 registers?

参考：R2, R6

题 13 | 什么情况下变量会 Spill 到 Local Memory？如何发现？

难度	★★☆	频率	极高	标签	Spill / 编译器

考察意图

考察从源码到 PTX/SASS 的资源诊断能力。

思路拆解

Spill 常由 register pressure 过高、动态索引的局部数组、过度 unroll、大量 live ranges 等触发。Local memory 访问可能命中 L1/L2，但本质上仍是 memory traffic。

标准回答

- 用 `nvcc -Xptxas=-v` 查看 registers 与 spill load/store。
- Nsight Compute 可观察 local memory instructions/traffic，并关联 source。
- 优化方法包括缩短 live range、减少不必要临时变量、调整 unroll、改变 tile、重新权衡 block size/register cap。

- 不要机械使用 `-maxrregcount`；它可能让 occupancy 上升但 spill 更严重。

Gotchas：常见失分点

- 把 spill 误认为 shared-memory spill。

面试官可能继续追问

- 动态索引为什么更容易阻止 registerization?
- 如何用 SASS 判断 local load/store?

参考：R2

题 14 | Shared Memory 为什么快？它解决的三个核心问题是什么？

难度	★☆☆	频率	极高	标签	Shared Memory / Reuse

考察意图

考察是否知道 shared memory 的真正价值不是“比 global 快”这句口号。

思路拆解

Shared memory 是程序员可控的片上 scratchpad。最典型的价值是：把不规则 global access 转成 coalesced load/store；缓存一个 tile 以消除重复 global load；在 block 内线程之间交换数据。

标准回答

- 显式 staging 可提高 data reuse 和 arithmetic intensity。
- 可以先 coalesced 地读入 shared，再按任意索引重排，例如 matrix transpose。
- 可以用于 reduction、scan、histogram 等 block 内协作。
- 代价包括额外 load/store 指令、barrier、bank conflict，以及 shared-memory capacity 对 occupancy 的限制。

Gotchas：常见失分点

- 把所有 global data 都搬 shared，忽略一次性读取的 copy/sync 成本。

面试官可能继续追问

- 什么时候 L1 已经足够，不值得手工 shared tiling?
- shared-memory tile 太大有什么副作用?

参考：R2

题 15 | L1 Cache 与 Shared Memory 的区别是什么？

难度	★★☆	频率	高	标签	Cache / Shared Memory

考察意图

考察硬件管理缓存和软件管理 scratchpad 的差异。

思路拆解

两者都在 SM 附近，但控制方式、语义和可预测性不同。L1 按硬件策略自动缓存，shared memory 由程序显式布局、显式同步、显式控制 lifetime。

标准回答

- L1 优点是透明，代码简单；缺点是 replacement、命中率不可完全控制。
- Shared memory 可精确安排 tile、swizzle、padding 和 producer/consumer，但要求处理 bank conflict 与同步。
- 对于规则的跨线程复用，shared 往往更可控；对于不值得 staging 的只读数据，cache 可能已经足够。
- 现代架构上 shared/L1 的物理资源组织会随架构变化，因此不要把某一代具体容量当永久规则。

Gotchas：常见失分点

- 认为 shared memory 就是“可编程 L1”的完全同义词。

面试官可能继续追问

- 怎么判断 shared staging 是否值得?
- L1 hit 很高时还需 shared 吗?

参考：R1, R2

题 16 | L2 Cache 为什么对多 Block / 多 SM 工作负载重要？

难度	★★☆	频率	中高	标签	L2 / 跨 SM

考察意图

考察 device-level 数据复用视角。

思路拆解

L1/shared 的复用范围偏 SM/block，而 L2 是 GPU 设备级共享层。不同 SM 上 blocks 访问共同只读数据、相邻 tiles 或 producer/consumer kernel 的中间数据时，L2 可能显著减少 HBM 流量。

标准回答

- L2 对跨 SM 的重复读取、kernel 间 temporal locality 很重要。
- 性能分析要同时看 L2 hit rate 和 DRAM bytes；高 L2 命中可能让“global memory”访问并不真正到 HBM。
- 数据布局、工作分块和 kernel launch 顺序都能影响 L2 locality。
- 若 working set 远大于 L2 或访问没有 temporal locality，不能依赖 L2“自动救场”。

Gotchas：常见失分点

- 把 global load 都当作 HBM transaction。

面试官可能继续追问

- 如何通过 tile ordering 提升 L2 locality?
- fusion 和 L2 reuse 的关系是什么?

参考：R2, R4

题 17 | Constant Memory 什么访问模式最快？

难度	★☆☆	频率	高	标签	Constant / Broadcast

考察意图

考察特殊 memory path 的访问模式。

思路拆解

Constant cache 特别适合一个 warp 中许多线程读取相同地址，因为硬件可广播。若 32 lanes 读取 32 个不同 constant addresses，访问可能被序列化，优势消失。

标准回答

- 适合小型只读参数、查表常量、每 warp 一致的标量。
- “只读”不是选择 constant memory 的充分条件；关键是 warp 内地址是否高度一致。
- 容量与 cache 资源有限，不适合大规模随机 lookup。

Gotchas：常见失分点

- 回答“所有只读数组都应该放 constant memory”。

面试官可能继续追问

- constant 与 `__ldg`/read-only cache 的差别?
- 每个 lane 不同地址时会发生什么?

参考：R2

题 18 | Texture / Read-Only Cache 什么时候有价值？

难度	★★☆	频率	中	标签	Texture / Cache

考察意图

考察是否能根据访问局部性选择 memory path。

思路拆解

Texture 路径历史上对二维局部性、不规则只读访问和硬件寻址/过滤非常有价值。现代通用 cache 已增强，因此是否使用 texture 应由数据类型、访问模式和 profile 驱动。

标准回答

- 二维 spatial locality 或采样/插值类 workload 是典型适用场景。
- 对完全连续、规整线性读取，普通 global load 通常已经很好。
- 不要为了“cache 更快”机械改成 texture；额外 API/绑定复杂度也有工程成本。

Gotchas：常见失分点

- 把 texture cache 当成比 L1 永远更快。

面试官可能继续追问

- 图像卷积和稀疏 gather 哪类更可能受益?
- 现代编译器/缓存下怎么 benchmark?

参考：R1, R2

题 19 | `cudaMalloc` 与 Unified Memory / `cudaMallocManaged` 怎么选？

难度	★★☆	频率	高	标签	Unified Memory / 系统

考察意图

考察显式数据管理与统一地址/迁移机制的权衡。

思路拆解

Managed memory 简化了可编程性，但页迁移、page fault、oversubscription 和 CPU/GPU 交替访问可能引入不可预测的成本。高性能场景需要理解实际 residency。

标准回答

- `cudaMalloc` + 显式 memcpy 给数据移动更强的可控性。
- Managed memory 提供统一地址空间和运行时迁移，适合开发效率或复杂数据结构。
- 可使用 prefetch/advice 等机制引导 residency；同时关注 NUMA 和 CPU/GPU 访问模式。
- 性能敏感代码不能只因为 API 简单就假设 Unified Memory 自动最优。

Gotchas：常见失分点

- 把 Unified Memory 说成“同一块物理内存，永远无需传输”。

面试官可能继续追问

- 什么是 page fault/migration?
- oversubscription 时会发生什么?

参考：R1, R2

题 20 | Pinned Host Memory 为什么能提高 H2D/D2H 吞吐？

难度	★☆☆	频率	极高	标签	Pinned Memory / PCIe

考察意图

考察 host-device 数据通路。

思路拆解

Page-locked/pinned host memory 能直接作为 DMA 传输的稳定源/目标，避免 pageable memory 额外 staging，且是实现真正异步拷贝与 copy/compute overlap 的常见前提。

标准回答

- Pinned memory 通常可提高 PCIe/NVLink host-device transfer throughput。
- 与 `cudaMemcpyAsync` 和多个 streams 配合可做 overlap。
- Pinned memory 是有限系统资源，过量 pin 会影响 OS 内存管理与整体系统性能。
- 小传输应考虑 batching；即使 bandwidth 高，频繁调用的 latency 仍可能主导。

Gotchas：常见失分点

- 把所有 host memory 都 pin。
- 忽略 NUMA affinity。

面试官可能继续追问

- 什么时候 zero-copy mapped pinned memory 有意义?
- 如何设计 chunked H2D-kernel-D2H pipeline?

参考：R2

本章复盘：面试前必须能说清的因果链

- 快不等于免费：register/shared 都会反向限制 residency。
- Local memory 是 thread-private 的 device-memory 地址空间，spill 是重点风险。
- Host-device 性能不仅看带宽，还看 pinned、NUMA、batching 与 overlap。

第三章 Global Memory 与 Coalescing

把 warp 访问模式翻译成 memory transactions。

本章抓手 transaction/sector · alignment/stride · effective vs actual bytes

题 21 | 什么叫 Global Memory Coalescing？

难度	频率	标签
★☆☆	极高	Coalescing / Global Memory

考察意图

这是 CUDA 优化的第一核心题：把 32 个线程的地址模式映射为实际 memory transactions。

思路拆解

Coalescing 不是简单说“线程地址连续”。正确分析要以 warp 为单位，看请求落在哪些对齐的 memory segments/sectors 上，并尽可能用最少 transaction 覆盖有用数据。

标准回答

- 对 compute capability 6.0+ 的常见简化：warp 请求会被服务成覆盖其地址所需的若干 32-byte transactions。
- 连续且对齐的 32 个 float load（128 B useful）理想情况下可由 4 个 32-B transactions 覆盖。
- stride、misalignment、稀疏 gather 都可能使 transferred bytes 远大于 requested bytes。
- 优化目标是提高 useful bytes / transferred bytes，而不是“看起来代码连续”。

Gotchas：常见失分点

- 逐 thread 分析，不把 32 lanes 放在一起。
- 认为只要相邻线程访问相邻元素就一定没有额外 transaction，忽略 alignment/边界。

面试官可能继续追问

- AoS 与 SoA 如何影响 coalescing?
- Nsight 中看哪些 sector/request 或 load efficiency 指标?

参考：R2

题 22 | 32 个线程连续读 32 个 float，理想需要多少 transaction？

难度	频率	标签
★☆☆	极高	口算 / Coalescing

考察意图

这是经典现场口算题，用来确认你真的理解 transaction，而不是会背概念。

思路拆解

32 float = 128 bytes。若起始地址和访问模式理想对齐，在常见 32-byte transaction 模型下，需要 4 个 transaction 覆盖 128 bytes。若跨边界或 misaligned，则可能需要更多。

标准回答

- 先算 useful bytes：32 × 4 = 128 B。
- 再按 32-B segment 覆盖：128 / 32 = 4。
- 强调“理想对齐”这个前提；不要把 4 当成所有架构、所有 cache path 的绝对物理事实。

Gotchas：常见失分点

- 忘记 alignment 条件。

面试官可能继续追问

- 如果从 `x[1]` 开始会怎样?
- 如果每个线程读 float4 呢?

参考：R2

题 23 | 什么是 Misaligned Access？现代 GPU 上为什么仍要关注？

难度	★★☆	频率	高	标签	Alignment / Global Memory

考察意图

考察地址边界、cache 与 transaction 效率。

思路拆解

Misalignment 可能让一个 warp 原本由 4 个 segments 覆盖的请求跨到第 5 个 segment。现代 cache/reuse 会缓和惩罚，但无效传输依旧存在，尤其在 bandwidth-bound kernel 中。

标准回答

- 先判断一个 warp 的地址跨度与 segment 边界。
- 再判断多出来的 sector 是否能被相邻 warp/cache 重用。
- 因此“理论多一个 transaction”不等于“性能一定按 25% 下降”；需要 profile actual DRAM/L2 traffic。

Gotchas：常见失分点

- 用早期 GPU 的固定 coalescing 规则套所有现代架构。

面试官可能继续追问

- 为什么相邻 warp 可能重用多读的 cache line?
- allocator alignment 有什么保证?

参考：R2

题 24 | 为什么 Strided Access 很危险？

难度	★☆☆	频率	极高	标签	Stride / Bandwidth

考察意图

考察 memory bandwidth 浪费。

思路拆解

若 warp 的 lane t 访问 `x[t * stride]`，stride 很大时每个 lane 可能落在不同 cache line/sector，导致大量 transaction 只承载很少 useful bytes。

标准回答

- stride=1 往往最容易 coalesce。
- 大 stride 会让 requested/actual byte ratio 变差，甚至一个 lane 占一个 sector。
- transpose、column-major/row-major 不匹配和结构体字段访问都是常见来源。
- shared-memory tiling 常用于把 strided global access 变成 coalesced access + 片上转置。

Gotchas：常见失分点

- 只说“cache miss 多”，不解释 transaction waste。

面试官可能继续追问

- 二维矩阵按列访问如何改?
- 为什么 shared memory 内 stride 不等于 global memory stride 的代价?

参考：R2

题 25 | AoS 与 SoA 哪个更适合 CUDA？

难度	★★☆	频率	极高	标签	Data Layout / Coalescing

考察意图

考察“布局跟随 warp access pattern”的思维。

思路拆解

不存在绝对答案。如果 warp 只读取一个字段，SoA 让该字段连续排列通常更容易 coalesce；如果每个线程马上需要一个对象全部字段，AoS/AoSoA 可能更自然。

标准回答

- SoA：字段分离，适合同一 warp 批量读取同一字段。
- AoS：对象连续，适合每线程消费完整结构，但可能把不需要的字段也带入 cache line。
- AoSoA / vectorized structure 是常见折中，用 tile/warp 宽度组织。
- 最终看 access pattern、alignment、cache line utilization 和下游计算。

Gotchas：常见失分点

- 把“GPU 必须 SoA”当定律。

面试官可能继续追问

- 什么时候 AoSoA 更好?
- 结构体 padding 会怎样影响 bandwidth?

参考：R2

题 26 | Vectorized Load/Store（如 float4）什么时候有收益？

难度	★★☆	频率	高	标签	Vectorization / 指令

考察意图

考察指令数、alignment 和 layout 的联合优化。

思路拆解

Vector load/store 可能减少 load/store 指令和 address calculation，但要求正确对齐与足够连续的数据。它不会自动修复糟糕的 warp address pattern。

标准回答

- 先保证 pointer alignment 与数据长度边界。
- vectorized access 主要优化 instruction overhead；memory transaction 是否更优仍由 warp 的地址集合决定。
- 尾部元素需要 scalar cleanup 或 masked handling。
- 向量化可能增加每线程 register 使用，需要一起看 occupancy。

Gotchas：常见失分点

- 认为 float4 会自动把 4 倍数据合并成 1 个 DRAM transaction。

面试官可能继续追问

- 128-bit load 对寄存器压力有什么影响?
- 编译器什么时候会自己生成宽 load?

参考：R2

题 27 | 为什么 Matrix Transpose 是经典 CUDA 优化题？

难度	★★☆	频率	极高	标签	Transpose / 综合

考察意图

它同时检查 global coalescing、shared-memory reordering、bank conflict 和 synchronization。

思路拆解

Naive transpose 常出现“读 coalesced、写 strided”或相反。经典解法用 shared tile：global 连续读 → shared 中转置 → global 连续写；随后再用 padding 处理 shared bank conflict。

标准回答

- 第一层优化：使用 32×32 或类似 tile，使读写都 coalesced。
- 第二层：shared tile 若按列读取可能造成 32-way bank conflict，典型用 `tile[32][33]` padding。
- 第三层：处理矩阵边界、block shape、每线程处理多元素以控制 block size。
- 最终以 effective bandwidth 与 copy kernel 上限对比，而不只看 absolute time。

Gotchas：常见失分点

- 只解决 global coalescing，忘了 bank conflict。

面试官可能继续追问

- 为什么 +1 padding 有效?
- 怎样设计非方阵/非 32 倍数版本?

参考：R2

题 28 | Effective Bandwidth 怎么计算？

难度	★☆☆	频率	高	标签	指标 / Bandwidth

考察意图

考察把算法数据量与时间换算成硬件可解释指标。

思路拆解

对于简单读写 kernel，effective bandwidth = (logical bytes read + logical bytes written) / time。它是“有用数据吞吐”，可与实际 DRAM throughput/理论峰值联合分析。

标准回答

- 例如 N 个 float 从 x 读、向 y 写：逻辑流量约 8N bytes。
- 如果 N=1e9，时间 10 ms，则 effective BW≈800 GB/s。
- 若 profiler 显示 DRAM 实际传输了远大于 8N bytes，说明访问效率差。
- 若 effective BW 已接近设备可达峰值，则优化重点应转为减少 bytes，而非让 memory subsystem 更忙。

Gotchas：常见失分点

- 把 cache 命中后的真实 DRAM bytes 与算法 logical bytes 混为一谈。

面试官可能继续追问

- 为什么 ECC、频率、power limit 会影响可达峰值?
- copy benchmark 如何建立机器实测 roof?

参考：R2, R4

题 29 | Requested Bandwidth 与 Actual DRAM Bandwidth 有什么区别？

难度	★★☆	频率	高	标签	Profiler / Memory

考察意图

考察是否会区分“忙”与“有效”。

思路拆解

一个 kernel 可能 DRAM 吞吐很高，却因为 scattered access 搬运大量无用 bytes。Requested 是程序逻辑需要的数据；actual 是 cache/DRAM 真正传输的数据。

标准回答

- requested/actual 比例低通常说明 transaction utilization 差。
- 高 DRAM utilization + 低 useful throughput，优先修访问模式或减少 bytes。
- L2 hit 会让部分 global requests 不到 DRAM，因此还要结合 L2 sectors/bytes。

Gotchas：常见失分点

- 看到 DRAM 95% 就宣布“已经最优”。

面试官可能继续追问

- 如何识别 over-fetch?
- sector miss 与 cache hit 如何共同解释?

参考：R4

题 30 | 一个 Memory-Bound Kernel 最常见的优化方向有哪些？

难度	★★☆	频率	极高	标签	Memory Bound / 方法论

考察意图

考察能否从 Roofline 出发给出系统策略。

思路拆解

如果瓶颈真的是内存带宽，最有效的方向通常不是继续提高 FLOPs，而是减少每个输出需要搬运的 bytes、提高复用并让传输更高效。

标准回答

- 改善 coalescing 和 alignment，减少 transaction waste。
- 使用 shared/register tiling 提高 reuse，提升 arithmetic intensity。
- 融合 kernels，消除中间 tensor 的 global store/load。

- 选择更低精度/压缩表示，减少字节数；在代价低时用 recomputation 换 memory traffic。
- 利用 L2/locality、vectorized load/store、异步 prefetch 隐藏剩余 latency。

Gotchas：常见失分点

- 把“memory-bound”理解成“global memory 很慢，所以换 shared”而不计算 reuse。

面试官可能继续追问

- 如何判断 fusion 会不会因 register pressure 反而变慢?
- recompute 何时值得?

参考：R2, R4

本章复盘：面试前必须能说清的因果链

- 所有 global-memory 分析先把 32 lanes 的地址写出来。
- 性能看 useful bytes / transferred bytes，不看“代码看起来连续”。
- Transpose 是 coalescing + shared reordering + bank conflict 的综合模板。

第四章 Shared Memory、Warp 与 Divergence

片上协作、bank conflict 与 warp-level 编程。

本章抓手 bank mapping · divergence · warp primitives

题 31 | Shared Memory Bank 是什么？

难度	频率	标签
★☆☆	极高	Bank / Shared Memory

考察意图

考察 shared-memory 并发访问的硬件组织。

思路拆解

Shared memory 被划分为多个可并行服务的 banks。对 CC 5.x+ 的常见模型，32 banks、连续 32-bit words 映射到连续 banks，与 32-lane warp 对齐。

标准回答

- 如果 warp 的 32 个请求落在 32 个不同 banks，可高度并行服务。
- 地址到 bank 的直观映射可近似理解为 `(address/4) mod 32`（针对 32-bit word 的常见情形）。
- 具体低层行为与访问宽度/架构有关，因此公式是分析起点而不是所有情况的完备模型。

Gotchas：常见失分点

- 把 bank 当成 cache line。

面试官可能继续追问

- 64-bit access 怎么分析?
- swizzle 为什么能替代简单 padding?

参考：R2

题 32 | 什么是 Bank Conflict？Broadcast 为什么是例外？

难度	频率	标签
★☆☆	极高	Bank Conflict / Shared Memory

考察意图

考察冲突定义的精确性。

思路拆解

同一 warp 的一次 shared-memory 请求若多个不同地址映射到同一 bank，硬件需要拆成多个 conflict-free 请求，降低有效带宽。但多个线程读取同一 shared address 可以广播，不按普通 conflict 处理。

标准回答

- 冲突的本质是“same bank + different addresses”。
- n-way conflict 可近似理解为该 bank 上请求被分成 n 个阶段服务。
- broadcast/multicast 是重要例外，所以“同 bank 就冲突”是错误答案。

Gotchas：常见失分点

- 把相同地址广播也算 32-way conflict。

面试官可能继续追问

- 原子操作在 shared bank 上如何影响性能?
- Nsight 如何显示 bank conflicts?

参考：R2

题 33 | 为什么 `tile[32][32]` 的列访问会出现 32-Way Bank Conflict？

难度	★★☆	频率	极高	标签	Transpose / Bank

考察意图

考察能否手算 bank mapping。

思路拆解

若二维数组按 row-major，每行 32 个 float。沿同一列访问时，相邻线程地址相差 32 words；`(base + 32*t) mod 32` 恒定，因此 32 lanes 映射到同一 bank 的不同地址。

标准回答

- stride=32 words 与 bank count=32 发生周期共振。
- 结果是同一 warp 的列读取被高度序列化。
- 这就是 transpose shared tile 的第二层优化问题。

Gotchas：常见失分点

- 只说“因为按列访问不连续”，没有算 bank。

面试官可能继续追问

- 如果维度是 31 或 33 会怎样?
- 如何用 gcd(stride, banks) 快速判断冲突模式?

参考：R2

题 34 | 为什么 `tile[32][33]` 能消除典型 Transpose Bank Conflict？

难度	★★☆	频率	极高	标签	Padding / Bank

考察意图

考察 padding 的数学原因。

思路拆解

加一列 padding 后行 stride 从 32 变成 33 words。由于 33 mod 32 = 1，相邻 lane 的列访问会依次落到 bank 0,1,...31，而不是都落同一 bank。

标准回答

- padding 改变的是 shared-memory address mapping，不是 global layout。
- 代价是少量额外 shared capacity，通常远小于冲突序列化收益。
- 更现代的高性能库也会使用 swizzled layouts，而不只依赖 +1 padding。

代码 / 白板骨架

```
constexpr int TILE = 32;
__shared__ float tile[TILE][TILE + 1]; // +1 改变列访问的 bank 映射

int x = blockIdx.x * TILE + threadIdx.x;
int y = blockIdx.y * TILE + threadIdx.y;
tile[threadIdx.y][threadIdx.x] = in[y * width + x];
__syncthreads();
// 之后按转置坐标读取 tile，避免 32-way bank conflict。
```

Gotchas：常见失分点

- 把 `[32][33]` 理解成改变矩阵逻辑形状。

面试官可能继续追问

- 为什么 CUTLASS/CuTe 常用 swizzle?
- padding 会影响 shared-memory occupancy 吗?

参考：R2

题 35 | 什么是 Warp Divergence？

难度	★☆☆	频率	极高	标签	Divergence / Warp

考察意图

考察控制流效率。

思路拆解

Divergence 是同一 warp 内 active lanes 需要执行不同控制流路径。硬件会分别执行这些路径并用 mask 屏蔽不参与的 lanes，所以有效 lane utilization 下降。

标准回答

- 关键是“同一 warp 内”是否分歧，而不是是否存在 branch。
- 若整个 warp 条件一致，即使不同 warps 走不同分支，也不会产生 warp 内 divergence。
- 长且代价相近的两条路径最容易造成明显浪费；短分支可能被 predication。

Gotchas：常见失分点

- “有 if 就 divergence”。

面试官可能继续追问

- 数据排序能否减少 divergence?
- divergence 与 memory coalescing 会如何耦合?

参考：R2, R8

题 36 | `if` 一定产生 Divergence 吗？如何重构控制流？

难度	频率	标签
★☆☆	高	Branch / Predication

考察意图

考察能否把条件表达式映射到 warp lane 分布。

思路拆解

例如条件以 warpId 为单位变化时，warp 内所有 lanes 一致，不产生 divergence。重构时可以通过数据分组、warp-granular mapping 或把短分支交给 predication，减少分歧。

标准回答

- 分析 predicate 在 32 lanes 上的分布。
- 尽量让同类数据由同一 warp 处理；必要时先 compaction/bucketing。
- 不要为了消 branch 引入大量额外 arithmetic 或 memory traffic。

Gotchas：常见失分点

- 用 branchless 技巧导致更多指令，但没有 benchmark。

面试官可能继续追问

- 什么时候 branchless 反而更慢?
- Nsight 哪些指标显示 branch efficiency?

参考：R2, R4

题 37 | Predication 是什么？为什么“消灭 if”不一定更快？

难度	★★☆	频率	中高	标签	Predication / Compiler

考察意图

考察编译器如何处理短控制流。

思路拆解

编译器可能把短分支转成 predicated instructions：指令被发射，但只有 predicate 为真的 lanes 写回结果。这样避免显式 branch/reconvergence，但会执行一些被屏蔽的指令。

标准回答

- 短、简单分支常适合 predication。
- 长路径用 predication 会执行太多无效工作，因此编译器可能保留 branch。
- 性能判断应看 SASS、executed instruction count、branch/warp execution efficiency。

Gotchas：常见失分点

- 把 branchless 写法当绝对优化。

面试官可能继续追问

- 如何从 SASS 看 predicate?
- compiler heuristics 受什么影响?

参考：R1, R4

题 38 | `__shfl_sync` 为什么可能比 Shared Memory 交换更快？

难度	★★☆	频率	极高	标签	Shuffle / Warp

考察意图

考察 warp register exchange。

思路拆解

Shuffle 允许 warp lanes 直接交换 register 值，避免“写 shared → barrier/sync → 读 shared”的额外路径，特别适合 reduction、scan、broadcast、small transpose。

标准回答

- 通信范围是 warp 内，因此不能替代跨 warp/block 通信。
- 必须给出正确 active mask，尤其是 partial warp。
- Volta+ 不应依赖隐式 warp lockstep；需要使用 `_sync` primitives。
- shuffle 会增加数据依赖链，极端情况下仍要关注 instruction latency/ILP。

Gotchas：常见失分点

- 对 partial warp 无脑传 `0xffffffff`。

面试官可能继续追问

- `__ballot_sync`、`__match_any_sync` 有什么用途?
- warp reduction 如何处理 non-power-of-two active lanes?

参考：R8

题 39 | `__syncwarp()` 什么时候必须用？

难度	★★☆	频率	高	标签	Warp Sync / 内存顺序

考察意图

考察 Volta+ independent thread scheduling 下的正确性。

思路拆解

如果 warp lanes 通过 shared/global memory 交换数据，且存在“写后读”依赖，不能只靠“warp 本来一起执行”的历史假设。`__syncwarp(mask)` 可同步指定 lanes，并提供相应 memory ordering。

标准回答

- shuffle/vote 自带其定义的同步语义，但 shared-memory 手工通信通常需要显式同步。
- mask 必须准确描述参与线程。
- warp sync 只覆盖 warp 范围；跨 warp 仍需 block/barrier 等更强机制。

Gotchas：常见失分点

- 认为 warp 永远隐式 lockstep，所以不需要 sync。

面试官可能继续追问

- 什么叫 warp-synchronous programming?
- independent thread scheduling 为什么会暴露旧代码 bug?

参考：R8, R7

题 40 | 如何写一个 Warp-Level Reduction？

难度	★★☆	频率	极高	标签	Reduction / Shuffle

考察意图

考察你能否把树形 reduction 映射为 5 次 shuffle。

思路拆解

32-lane warp 的和可以按 offset 16、8、4、2、1 做 `__shfl_down_sync`，每轮让低 lane 累加来自更高 lane 的值。完整 block reduction 通常先每 warp reduction，再把 warp sums 合并。

标准回答

- 32 个值归约到 1 个需要 log2(32)=5 轮。
- partial warp 要用 `__activemask()` 或 `__ballot_sync` 构造有效 mask。
- 对于 floating-point sum，树形顺序与串行顺序不同，数值结果可能有舍入差异。

代码 / 白板骨架

```
__device__ float warp_sum(float x, unsigned mask) {
    for (int offset = 16; offset > 0; offset >>= 1)
        x += __shfl_down_sync(mask, x, offset);
    return x;
}
```

Gotchas：常见失分点

- 只会写 shared-memory reduction。

面试官可能继续追问

- 如何做 max/argmax?
- 数值稳定性要求更高时如何改?

参考：R8

本章复盘：面试前必须能说清的因果链

- Bank conflict 是 same bank + different addresses；same address broadcast 是例外。
- Divergence 看 warp 内 lane 路径，而不是看源代码有没有 if。
- Warp shuffle 是 register exchange，但 partial warp 和 mask 必须正确。

第五章 Occupancy、 Register 与 Latency Hiding

资源约束、TLP/ILP 与“高 occupancy 迷思”。

本章抓手 occupancy/resource equations · register pressure · TLP vs ILP

题 41 | 什么是 Occupancy？

难度	★☆☆	频率	极高	标签	Occupancy / 资源

30 秒答案 Occupancy 只回答“SM 上有多少 active warps”，不是“这些 warps 做得是否高效”。

考察意图

高频陷阱题：考察你是否知道 occupancy 只是 latency-hiding 的代理变量。

思路拆解

Occupancy = active warps per SM / theoretical maximum warps per SM。它说明有多少 warp 状态可用于调度，但不能直接代表算术效率、memory efficiency 或最终性能。

标准回答

- 低 occupancy 往往削弱隐藏 latency 的能力。
- 高 occupancy 超过“足够隐藏 latency”的点后，继续提高未必增益。
- register、shared memory、threads/block、block slots 等都可能限制 occupancy。
- 官方 Best Practices 明确指出 higher occupancy does not always mean higher performance。

Gotchas：常见失分点

- 把 occupancy 当目标函数。

面试官可能继续追问

- 为什么 50% occupancy 的 GEMM 可能比 100% 更快?
- achieved occupancy 与 theoretical occupancy 有何区别?

参考：R2

题 42 | 为什么 100% Occupancy 不一定最快？

难度	★★☆	频率	极高	标签	Occupancy / ILP

考察意图

考察 TLP 与每线程资源之间的 trade-off。

思路拆解

为了追求 100% occupancy 强行降低 registers/thread，可能造成 spill、缩小 register tile、降低 ILP 或增加指令数。某些高算术强度 kernel 在较低 occupancy 下仍能用 ILP 覆盖延迟。

标准回答

- 方案 A：更多 warps，但每线程资源少。
- 方案 B：更少 warps，但每线程有更大 register tile、更少 memory traffic、更高 ILP。
- 性能取决于瓶颈；当 latency 已被隐藏时，继续增加 warps 不会线性提速。

Gotchas：常见失分点

- 为了 occupancy 使用 `-maxrregcount`，不检查 spill。

面试官可能继续追问

- 如何人为增加 dynamic shared memory 来做 occupancy sensitivity 实验?
- 什么时候 occupancy 确实是主要瓶颈?

参考：R2

题 43 | 哪些资源会限制 Resident Blocks / Occupancy？

难度	频率	标签
★★☆	极高	资源模型 / SM

考察意图

考察是否会算 occupancy 约束。

思路拆解

每个 SM 能驻留多少 blocks 由多个上限共同决定，取最紧的那个：threads、warps、register file、shared memory、最大 blocks/SM 以及架构特定限制。

标准回答

- thread/block × blocks ⩽ max threads/SM。
- warp/block × blocks ⩽ max warps/SM。
- register allocation 与 shared-memory allocation 必须满足 SM 总资源。
- 即使 thread 数很少，block slot 上限也可能成为约束。

Gotchas：常见失分点

- 只算 threads，不算 register/smem。

面试官可能继续追问

- 如何使用 occupancy API/Calculator?

- register allocation granularity 为什么会造成“少 1 个 register 突然多一个 block”？

参考：R2

题 44 | 什么是 Register Pressure？如何权衡？

难度	★★☆	频率	极高	标签	Register / Occupancy

考察意图

考察高性能 kernel 最常见资源矛盾。

思路拆解

Register pressure 是 kernel 对 per-thread registers 的需求偏高，导致 resident warps/blocks 降低，甚至 spill。高性能优化不是简单降 register，而是寻找 registers、ILP、reuse 与 occupancy 的平衡点。

标准回答

- 扩大 thread tile、unroll、fusion 都可能增加 register live ranges。
- 适度高 register 能减少 local/global traffic，并提高 accumulator reuse。
- 过高会限制 occupancy；超过可分配范围还会 spill。
- 应结合 ptxas 输出、Nsight occupancy、local-memory traffic 一起判断。

Gotchas：常见失分点

- 只看源代码变量数量估算 register。

面试官可能继续追问

- 如何缩短 live range?
- 为什么 fusion 常把 memory-bound 变成 register-bound?

参考：R2

题 45 | 如何查看 Kernel 使用多少 Registers / Shared Memory / Spills？

难度	★☆☆	频率	极高	标签	工具 / 编译

考察意图

考察基本工具链。

思路拆解

编译阶段先用 ptxas verbose 查看静态资源；运行阶段用 Nsight Compute 看 achieved occupancy、local memory、spill 相关访问、动态 shared memory 与 source/SASS。

标准回答

- `nvcc -Xptxas=-v` 查看 registers、stack、spill stores/loads 等。
- Nsight Compute 的 Launch Statistics/Occupancy 等 sections 可检查 block resource constraints。
- 必要时查看 PTX/SASS 和 source correlation，定位是哪个代码段延长 live range。

Gotchas：常见失分点

- 只依赖 `nvcc` 优化级别猜测。

面试官可能继续追问

- 为什么 debug build 的 resource usage 不能代表 release?
- 如何验证一次源码改动是否减少 spill?

参考：R2, R4

题 46 | `-maxrregcount` 是不是越小越好？

难度	★★☆	频率	高	标签	Compiler / Register

考察意图

考察是否会避免“为了 occupancy 限寄存器”的过度优化。

思路拆解

限制 register count 可能提高理论 occupancy，但编译器为了满足上限会增加 local memory spill 或更多指令。必须用 end-to-end benchmark 验证。

标准回答

- 先确认当前 kernel 是否真的 latency/occupancy-bound。
- 若已经 compute-bound 或 memory-bandwidth-bound，强行提 occupancy 可能没有收益。
- 逐档测试 register cap，并同时记录 spill bytes、occupancy、kernel time。

Gotchas：常见失分点

- 把 occupancy calculator 的最高点当自动调参目标。

面试官可能继续追问

- `__maxnreg__` 与 `__launch_bounds__` 有何关系?
- 为什么两者不能随意混用?

参考：R7, R2

题 47 | `__launch_bounds__` 用来解决什么问题？

难度	★★☆	频率	高	标签	Launch Bounds / Compiler

考察意图

考察编译器资源约束提示。

思路拆解

`__launch_bounds__(maxThreadsPerBlock, minBlocksPerMultiprocessor)` 向编译器声明 launch 上界/期望 residency，使其在 register 分配与指令数之间做更合适的权衡。

标准回答

- 第一参数可防止编译出在目标 block size 下资源过量的 kernel，并帮助前向兼容。
- 第二参数会影响 compiler 推导 register 上限，试图允许至少指定数量 blocks resident。
- 编译器有时会主动增加 register 使用来减少指令，只要不破坏给定 bounds。
- 它不是“保证性能”的 annotation，仍需 profile。

Gotchas：常见失分点

- 把 `minBlocksPerMultiprocessor` 当运行时硬保证。

面试官可能继续追问

- 为什么 bound 太激进会增加 local memory?
- 不同 architecture 是否需要不同 bounds?

参考：R7, R2

题 48 | ILP 与 TLP 有什么区别？

难度	★★☆	频率	极高	标签	ILP / TLP

考察意图

考察延迟隐藏的两条路线。

思路拆解

TLP 依靠更多 warps/threads 在一个 warp 等待时切换；ILP 依靠同一 thread/warp 中多个独立指令，让流水线有别的工作可发射。二者常与 register 使用互相制约。

标准回答

- TLP 的代理指标包括 occupancy、eligible warps。
- ILP 常来自 loop unroll、multiple accumulators、software pipelining。
- 增加 ILP 往往增加 registers；增加 TLP 往往要求控制 per-thread resources。
- 高性能 GEMM 通常不是追求最大 TLP，而是用很强 ILP + 足够 TLP。

Gotchas：常见失分点

- 把 unroll 只理解成减少 loop overhead。

面试官可能继续追问

- 为什么 multiple accumulators 能隐藏 FMA latency?
- 如何从 scheduler metrics 判断缺 ILP 还是缺 warps?

参考：R2, R4

题 49 | GPU 如何用 Warp Scheduling 隐藏 Memory Latency？

难度	★★☆	频率	极高	标签	Latency Hiding / Scheduler

考察意图

考察 GPU throughput 架构的核心机制。

思路拆解

一个 warp 发出高延迟 load 后，如果后续指令依赖结果，它会进入等待；scheduler 可从其他 eligible warps 发射指令。如果没有足够可运行 warp 或独立指令，latency 就暴露为 stall。

标准回答

- resident warp 多不等于 eligible warp 多；如果所有 warps 都等同一内存依赖，occupancy 再高也可能一起 stall。
- ILP 可以让同一 warp 在等待一个 load 时继续执行不依赖该 load 的工作。
- software prefetch/pipeline 的本质就是更早发出 load，把等待期塞进有用计算。

Gotchas：常见失分点

- 认为 warp switch 有 CPU context switch 那样的高成本。

面试官可能继续追问

- long scoreboard 高说明什么?
- 怎样把 load 提前几个迭代?

参考：R2, R4

题 50 | Block Size 应该如何系统调优，而不是猜 256？

难度	★★☆	频率	极高	标签	Auto-tuning / 方法论

考察意图

考察实验设计。

思路拆解

先用资源模型排除不可能的配置，再用代表性 workload benchmark 64/128/256/512 等候选，并记录性能与资源指标。对二维 kernel 还要调 block shape。

标准回答

- 建立 baseline 和 correctness check。
- 查看 registers/thread、smem/block、theoretical occupancy。
- 测试多个 block size；对 memory kernel 同时关注 coalescing，对 compute kernel 关注 ILP/issue。
- 用 Nsight 验证性能变化的因果：是 occupancy、cache、stall 还是 instruction count 变化。

Gotchas：常见失分点

- 只在一个很小输入上调 block size。

面试官可能继续追问

- 为什么最佳 block size 会随 input shape 改变?
- 什么时候需要真正的 auto-tuner?

参考：R2, R4

本章复盘：面试前必须能说清的因果链

- Occupancy 是 latency-hiding 的手段，不是目标。
- Register pressure 同时影响 ILP、spill 与 occupancy。
- 性能要平衡 TLP 与 ILP，而不是单指标最大化。

第六章 经典 CUDA Kernel 现场编程

Reduction、Scan、Histogram、Transpose、GEMM 等必练题。

本章抓手 现场 coding · 逐版优化 · 正确性与边界

题 51 | 现场手写一个高性能 Reduction：你会如何逐版优化？

难度	★★★	频率	极高	标签	Coding / Reduction

30 秒答案 Reduction 的优化主线：减少 global atomics、减少 barriers/shared traffic、提高每线程工作量，最后用 warp shuffle。

考察意图

公开 CUDA 面经可核验的典型现场题。重点不是一次写出终极版，而是展示“发现瓶颈 → 逐版优化”的能力。

思路拆解

建议像《剑指 Offer》式按版本讲：global atomic 基线 → shared tree → sequential addressing → 每线程多元素 → warp shuffle → vectorized/global grid reduction。每一步说明减少了什么成本。

标准回答

- V0：每元素 global atomic，正确但 contention 大。
- V1：每 block 在 shared 中归约，只让一个 thread global atomic。
- V2：使用 sequential addressing，减少 interleaved divergence 与地址运算。
- V3：每 thread 先累加多个元素，提高 load 吞吐与 arithmetic/loop efficiency。
- V4：最后一个 warp 用 shuffle，减少 shared traffic 和 block barriers。
- V5：大规模场景用两阶段 kernel、cooperative groups 或合适的 device-wide primitive；工程上优先考虑 CUB。

代码 / 白板骨架

```
template<int BLOCK>
__global__ void reduce_sum(const float* x, float* out, int n) {
    float v = 0.f;
    for (int i = blockIdx.x * BLOCK * 2 + threadIdx.x;
         i < n; i += gridDim.x * BLOCK * 2) {
        v += x[i];
        if (i + BLOCK < n) v += x[i + BLOCK];
    }
    unsigned mask = __activemask();
    for (int off = 16; off > 0; off >>= 1)
        v += __shfl_down_sync(mask, v, off);

    __shared__ float warp_sum[32];
```

```
    int lane = threadIdx.x & 31;
    int wid  = threadIdx.x >> 5;
    if (lane == 0) warp_sum[wid] = v;
    __syncthreads();

    if (wid == 0) {
        float s = lane < (BLOCK + 31) / 32 ? warp_sum[lane] : 0.f;
        unsigned m = __ballot_sync(0xffffffff, lane < (BLOCK + 31) / 32);
        for (int off = 16; off > 0; off >>= 1)
            s += __shfl_down_sync(m, s, off);
        if (lane == 0) atomicAdd(out, s);
    }
}
```

Gotchas：常见失分点

- 错误地把 `__syncthreads()` 放进只有部分线程执行的分支。
- 对 float 求和忽略非结合性和数值稳定性。

面试官可能继续追问

- 如何做 double/half/bfloat16 reduction?
- 十亿 float 求和怎样提高精度?

参考：R2, R8, R12

题 52 | 为什么最原始 Interleaved Reduction 很慢？

难度	★★☆	频率	高	标签	Reduction / Divergence

考察意图

考察能否分析经典坏实现。

思路拆解

像 `if (tid % (2*s) == 0)` 的 interleaved 归约会让 active lanes 呈稀疏分布，产生 divergence，并包含 modulo/地址运算；随着 s 增大，warp utilization 进一步下降。

标准回答

- 改成 sequential addressing：`if (tid < s) data[tid] += data[tid+s]`。
- 避免 bank conflict/非必要同步，并让 active lanes 尽量连续。
- 进一步把最后一个 warp 改为 shuffle。

Gotchas：常见失分点

- 只说“modulo 慢”，忽略最重要的 active lane 利用率。

面试官可能继续追问

- 为什么 sequential addressing 更好?

- 最后 32 个元素还需要 block barrier 吗?

参考：R2

题 53 | Reduction 最后一 Warp 为什么适合用 Shuffle？

难度	★★☆	频率	高	标签	Reduction / Warp

考察意图

考察 block-level 到 warp-level 的分层优化。

思路拆解

归约到 <=32 个值后，数据交换范围已局限在一个 warp。继续用 shared memory + `__syncthreads()` 会多出 store/load/barrier；shuffle 可直接 register-to-register 交换。

标准回答

- 5 次 `shfl_down` 即可把 32 lanes 归约。
- partial warp 使用有效 mask。
- 对于多 warp block，可先每 warp 得一个 sum，写 shared，再由第一个 warp 归约 warp sums。

Gotchas：常见失分点

- 把 `__syncthreads()` 留在单 warp loop 中并认为“反正很快”。

面试官可能继续追问

- warp sum 写 shared 后为什么仍需要一次 block barrier?
- 可以用 cooperative groups 重写吗?

参考：R8

题 54 | 手写 Parallel Scan / Prefix Sum：Hillis-Steele 与 Blelloch 如何比较？

难度	★★★	频率	高	标签	Coding / Scan

考察意图

考察经典并行算法与 work efficiency。

思路拆解

Hillis-Steele 易写但做 O(N log N) work；Blelloch 采用 up-sweep + down-sweep，work O(N)，更适合构建高效 block scan。实际工程优先 CUB BlockScan/DeviceScan。

标准回答

- Hillis-Steele 每轮 offset 加倍，轮数 O(log N)，但每轮很多元素做加法。

- Blelloch up-sweep 构建 reduction tree，down-sweep 传播 prefix，可实现 exclusive scan。
- 大数组通常分块：block scan + block sums scan + uniform add。
- shared-memory 实现需要处理 bank conflict；warp scan 可用 shuffle 优化。

代码 / 白板骨架

```
// Warp inclusive scan 核心片段
unsigned mask = __activemask();
for (int offset = 1; offset < 32; offset <<= 1) {
    float y = __shfl_up_sync(mask, x, offset);
    if ((threadIdx.x & 31) >= offset) x += y;
}
```

Gotchas：常见失分点

- 把 inclusive/exclusive scan 混淆。
- 忘记多 block 的 block sums 二次扫描。

面试官可能继续追问

- scan 为什么是 stream compaction 的核心?
- segmented scan 怎么做?

参考：R8

题 55 | Parallel Histogram 如何优化 Atomic Contention？

难度	★★★	频率	极高	标签	Coding / Histogram / Atomic

考察意图

考察 privatization、shared atomics 与数据分布。

思路拆解

Naive `atomicAdd(global_hist + bin)` 在 bins 少或分布偏斜时会有严重 contention。典型优化是 block-private 或 warp-private histogram，在 shared memory 先聚合，再合并到 global。Hopper 还可进一步讨论 DSM。

标准回答

- 先判断 bins 数、分布 skew、数据类型和 shared capacity。
- block-private：每 block 的 shared histogram 用 shared atomics，最后少量 global atomics。
- 若 shared histogram 太大，可做 tiled bins、warp-private 分区或 CC 9.0+ DSM 中间层。
- 如果 bins 非常多、几乎无冲突，直接 global atomics 未必差。

代码 / 白板骨架

```
extern __shared__ unsigned smem_hist[];
for (int b = threadIdx.x; b < bins; b += blockDim.x) smem_hist[b] = 0;
__syncthreads();
```

```
for (int i = blockIdx.x * blockDim.x + threadIdx.x;
     i < n; i += blockDim.x * gridDim.x) {
    atomicAdd(&smem_hist[data[i]], 1);
}
__syncthreads();
for (int b = threadIdx.x; b < bins; b += blockDim.x)
    if (smem_hist[b]) atomicAdd(&global_hist[b], smem_hist[b]);
```

Gotchas：常见失分点

- 说“atomic 一定慢”。
- 私有 histogram 太大导致 shared memory 把 occupancy 压死。

面试官可能继续追问

- 极端 hot bin 怎么优化?
- DSM histogram 相比 global atomics 的限制是什么?

参考：R1, R5

题 56 | 现场写 Optimized Matrix Transpose。

难度	★★★	频率	极高	标签	Coding / Transpose

考察意图

考察你能否把 coalescing 与 bank conflict 连起来。

思路拆解

标准思路：每 block coalesced 读取一个 tile 到 shared；block barrier；交换 block 坐标并从 shared 的转置位置读取；coalesced 写出。Shared tile 使用 +1 padding 或 swizzle。

标准回答

- 用 32×8 threads 处理 32×32 tile，可让每线程循环处理 4 行，控制 block threads=256。
- global load/store 都按 x 连续，获得 coalescing。
- `tile[TILE][TILE+1]` 避免列读 bank conflict。
- 边界版本需要 x<width、y<height 的 guards，但 barrier 必须保持 block 一致。

代码 / 白板骨架

```
constexpr int TILE=32, ROWS=8;
__global__ void transpose(float* out, const float* in, int w, int h) {
    __shared__ float tile[TILE][TILE+1];
    int x = blockIdx.x*TILE + threadIdx.x;
    int y = blockIdx.y*TILE + threadIdx.y;
    for (int j=0; j<TILE; j+=ROWS)
        if (x<w && y+j<h) tile[threadIdx.y+j][threadIdx.x] = in[(y+j)*w+x];
```

```
    __syncthreads();
    x = blockIdx.y*TILE + threadIdx.x;
    y = blockIdx.x*TILE + threadIdx.y;
    for (int j=0; j<TILE; j+=ROWS)
        if (x<h && y+j<w) out[(y+j)*h+x] = tile[threadIdx.x][threadIdx.y+j];
}
```

Gotchas：常见失分点

- 在第一次 bounds if 内放 `__syncthreads()`。

面试官可能继续追问

- 为什么 32×8 常见?
- 如何测 transpose 是否接近 copy bandwidth?

参考：R2

题 57 | 手写 Tiled Matrix Multiplication，并解释为什么快。

难度	★★★	频率	极高	标签	Coding / GEMM

考察意图

考察 shared tiling、reuse 与 arithmetic intensity。

思路拆解

Naive GEMM 每个 C 元素反复从 global 读取 A/B。Tiled GEMM 让一个 block 先把 A/B 的 K 维片段读入 shared，每个元素被 block 内多个 threads 重用，再推进下一个 K tile。

标准回答

- global bytes 被多个 FMA 摊薄，arithmetic intensity 上升。
- load tile 后需要 block synchronization，确保 shared 数据完整。
- 每 thread 做多个 outputs/register tiling 可进一步提高 reuse 和 ILP。
- 现代高性能 GEMM 会继续分成 CTA tile、warp tile、MMA tile、thread/register tile，并使用 Tensor Core。

代码 / 白板骨架

```
template<int T>
__global__ void gemm(const float* A, const float* B, float* C, int N) {
    __shared__ float As[T][T], Bs[T][T];
    int row = blockIdx.y*T + threadIdx.y;
    int col = blockIdx.x*T + threadIdx.x;
    float acc = 0.f;
    for (int k0=0; k0<N; k0+=T) {
        As[threadIdx.y][threadIdx.x] = A[row*N + k0 + threadIdx.x];
        Bs[threadIdx.y][threadIdx.x] = B[(k0 + threadIdx.y)*N + col];
        __syncthreads();
        #pragma unroll
```

```
        for (int k=0;k<T;++k) acc += As[threadIdx.y][k] * Bs[k][threadIdx.x];
        __syncthreads();
    }
    C[row*N+col] = acc;
}
```

Gotchas：常见失分点

- 只做 shared tile，但 global loads 没 coalesce。
- tile 越大越好。

面试官可能继续追问

- 如何选择 BM/BN/BK?
- 为什么现代 GEMM 需要 double buffering?

参考：R2, R6

题 58 | 为什么 GEMM Tile 越大不一定越快？

难度	★★★	频率	高	标签	GEMM / 资源

考察意图

考察 reuse 与 occupancy/register pressure 的约束优化。

思路拆解

更大的 tile 通常提高 A/B reuse，减少 global bytes/FLOP；但也消耗更多 shared memory、registers 和线程资源，可能减少 active CTAs/warps，增加 spill 或降低 scheduler flexibility。

标准回答

- tile shape 要匹配 M/N/K 与硬件 Tensor Core/MMA shape。
- K tile 影响 pipeline depth 和 shared bytes；M/N tile 影响 accumulator registers。
- 边界/小矩阵还会出现 tile waste。
- 高性能库通常根据 architecture、dtype、shape 提供多个 kernel schedules，而不是一个万能 tile。

Gotchas：常见失分点

- 只追求最大 arithmetic intensity。

面试官可能继续追问

- 为什么 skinny GEMM 需要不同 schedule?
- Stream-K 解决什么问题?

参考：R6

题 59 | 如何实现 Stream Compaction？

难度	★★☆	频率	高	标签	Coding / Scan

考察意图

考察 predicate→scan→scatter 的标准并行模式。

思路拆解

先为每个元素计算 keep flag（0/1），对 flags 做 exclusive scan 得到每个保留元素的目标位置，再并行 scatter。若保留比例很低，也可讨论 warp ballot/atomic chunk allocation。

标准回答

- flags[i] = predicate(x[i])。
- pos = exclusive_scan(flags)。
- 若 flags[i]=1，则 out[pos[i]]=x[i]。
- 输出总数 = pos[N-1]+flags[N-1]。
- 工程上可使用 CUB DeviceSelect；自写时重点是 scan 和 memory traffic。

Gotchas：常见失分点

- 每个保留元素直接 global atomic 申请位置而不讨论 contention。

面试官可能继续追问

- warp ballot 如何做小规模 compaction?
- 稳定 compaction 与不稳定 compaction 的区别?

参考：R8

题 60 | Top-K / Sort / Selection 什么时候不应该自己写 CUDA？

难度	★★☆	频率	极高	标签	工程 / 库

考察意图

考察工程判断：高性能不是“所有东西手写”。

思路拆解

成熟库通常已经包含架构特化、边界处理和大量调优。面试中先确认 CUB/Thrust/cuBLAS/cuDNN/CUTLASS 是否满足语义与性能，只有在数据分布、融合机会或定制约束明显时才考虑自写。

标准回答

- CUB 对 scan/reduce/select/sort 等 primitives 很成熟。
- cuBLAS/cuBLASLt 对 GEMM 有自动算法选择与 epilogue。
- CUTLASS 适合需要定制 layout/epilogue/schedule 的 GEMM 家族。

- 自写 kernel 的成本包括正确性、跨架构维护、数值稳定性和回归测试。

Gotchas：常见失分点

- 把“会 CUDA”理解成拒绝任何 library。

面试官可能继续追问

- 何时 fused top-k 值得自写?
- 如何验证自写 kernel 真正优于库?

参考：R6

本章复盘：面试前必须能说清的因果链

- 现场 coding 先给正确 baseline，再逐版优化，解释每一步减少什么成本。
- 边界条件、barrier 一致性、数值误差是面试常见追杀点。
- 工程上能用 CUB/cuBLAS 等成熟库时应优先库，再讨论定制 kernel。

第七章 Stream、异步拷贝与 Software Pipeline

从“能并行”升级到“让搬运与计算持续重叠”。

本章抓手 stream overlap · async copy · TMA/pipeline

题 61 | CUDA Stream 是什么？不同 Stream 真的能并发吗？

难度	频率	标签
★☆☆	极高	Stream / 并发

考察意图

考察异步执行模型。

思路拆解

Stream 是有序的 GPU work queue。同一 stream 中操作按顺序建立依赖；不同 streams 之间在没有显式依赖且硬件资源允许时，可以并发或 overlap。

标准回答

- 不同 stream 并发不是保证：如果一个 kernel 已占满所有 SM/resource，另一个可能没有空间。
- copy/compute overlap 还依赖 copy engines、pinned memory、方向和平台能力。
- default stream 的语义需明确，避免无意的全局依赖。
- events 可在 streams 之间表达依赖，而不做 device-wide barrier。

Gotchas：常见失分点

- “两个 stream 就一定同时跑”。

面试官可能继续追问

- 如何检查 concurrent kernels?
- stream priority 有何作用?

参考：R1, R10

题 62 | `cudaMemcpyAsync` 为什么不一定真的异步？

难度	频率	标签
★★☆	极高	Memcpy / Pinned

考察意图

考察 API 名字与实际数据路径的区别。

思路拆解

异步拷贝要结合 host memory 类型、stream 和设备 copy engine 来判断。Pageable host memory 可能需要额外 staging；常见高吞吐 overlap 方案使用 pinned host buffers。

标准回答

- Pinned + non-default stream 是常见前提。
- 小 copy 的 API/PCIe latency 可能主导，因此 batching 很重要。
- 同一资源上的依赖会阻止 overlap。
- 要在 Nsight Systems timeline 中验证实际 overlap，而不是凭 API 名称判断。

Gotchas：常见失分点

- 把 `Async` 当语义上的绝对并发保证。

面试官可能继续追问

- H2D 与 D2H 能否同时发生?
- 为什么 pageable memory 需要 staging?

参考：R2, R10

题 63 | 如何设计 H2D - Kernel - D2H 的流水重叠？

难度	★★☆	频率	极高	标签	Pipeline / Streams

考察意图

考察 chunking 与 double buffering。

思路拆解

把输入分成 chunks，用 2-3 个 streams 和对应 pinned buffers，让 chunk i 的 kernel 与 chunk i+1 的 H2D、chunk i-1 的 D2H 重叠，形成 steady-state pipeline。

标准回答

- chunk 太小：launch/copy latency 占比高；太大：并发阶段少且 buffer 占用大。
- 每个 buffer 的生命周期必须与对应 stream/event 对齐，避免 CPU 提前覆写。
- pipeline 性能上限取决于最慢 stage；理想 steady-state throughput 接近 max(H2D, compute, D2H) 而不是三者之和。

Gotchas：常见失分点

- 只创建 streams，却复用同一个 host buffer 导致隐含等待或数据 race。

面试官可能继续追问

- 如何选择 chunk size?
- NVLink/PCIe 下策略有何差异?

参考：R2, R10

题 64 | CUDA Event 与 `cudaDeviceSynchronize()` 有什么区别？

难度	频率	标签
★☆☆	极高	Event / 同步

考察意图

考察细粒度依赖管理。

思路拆解

`cudaDeviceSynchronize()` 是粗粒度设备等待；event 可在某 stream 记录完成点，host 等 event 或其他 stream `cudaStreamWaitEvent`，从而表达局部依赖并保留其他并发。

标准回答

- GPU timing 通常用 events 记录同一 stream 的 start/stop。
- 跨 stream producer-consumer 用 event 建立 DAG，而不全设备同步。
- 频繁 device-wide synchronize 会破坏 pipeline 和 CPU/GPU overlap。

Gotchas：常见失分点

- 每个 kernel 后都 `cudaDeviceSynchronize()`。

面试官可能继续追问

- event timing 为什么通常比 CPU clock 更可靠?
- Graph 中依赖又如何表达?

参考：R10

题 65 | Ampere 的 Async Global→Shared Copy 解决什么问题？

难度	频率	标签
★★★	高	cp.async / Pipeline

考察意图

考察现代数据搬运。

思路拆解

传统 staging 常走 global load → register → shared store；async copy 可以把 global→shared 作为异步搬运，与计算重叠，并减少中间 register 路径。

标准回答

- 核心收益一：更早发起下一 tile load，隐藏 memory latency。
- 核心收益二：减少中间 register 占用，有时缓解 register pressure。
- 需要 pipeline/barrier 正确管理 stage 何时可读、何时可重用。

- 实际收益依赖 kernel 是否有足够计算来覆盖 copy latency。

Gotchas：常见失分点

- 只把同步 memcpy 换成 async API，却不改流水结构。

面试官可能继续追问

- `cp.async` 与 Hopper TMA 的根本差别?
- 需要多少 pipeline stages?

参考：R2, R13

题 66 | 什么是 Software Pipelining？

难度	★★★	频率	极高	标签	Software Pipeline / 真题

30 秒答案 Software pipelining = 把不同迭代的 load/compute 重叠，用额外 buffer/stage 换 latency hiding。

考察意图

这是公开 NVIDIA 面经可核验的现代 CUDA 题。考察你能否把循环迭代拆成多个重叠阶段。

思路拆解

在 tiled GEMM 等 K-loop 中，不再“load tile i → compute i → load tile i+1”，而是提前预取 future tile，使 memory stage 与 current compute 同时进行。

标准回答

- 把循环拆成 prologue、steady state、epilogue。
- 需要至少双缓冲：一个 buffer 被 consumer 计算，另一个 buffer 被 producer 填充。
- 更深 pipeline 可进一步隐藏 latency，但会消耗更多 shared/register resources。
- Software pipeline 的目标是减少 exposed latency，而不是减少 total bytes。

代码 / 白板骨架

```
// 概念时序:
// preload tile 0
// for k:
//   async_load(tile[k+1] -> buffer[next])
//   compute(buffer[cur])
//   wait/commit next stage
//   swap(cur, next)
```

Gotchas：常见失分点

- 只描述 CPU 编译器的 loop software pipelining，不映射到 GPU data movement。

面试官可能继续追问

- 如何确定 stage count?
- 为什么 pipeline 太深会变慢?

参考：R12, R2, R6

题 67 | 什么是 Double Buffering / Multi-Stage Pipeline？

难度	★★★	频率	高	标签	Double Buffer / Pipeline

考察意图

考察对 software pipeline 的资源实现。

思路拆解

双缓冲使用 A/B 两份 shared tile：计算 A 时加载 B，切换后计算 B 并加载下一 A。多 stage 进一步增加 in-flight tiles，以覆盖更长 latency。

标准回答

- buffer 数决定能同时在途的阶段数。
- stage 增多会增加 shared capacity，占用可能下降。
- producer/consumer 的 commit/wait 和 buffer reuse 必须严格同步。
- 最佳 stage count 取决于 latency、compute per tile、tile size 和架构。

Gotchas：常见失分点

- 认为 stage 越多越快。

面试官可能继续追问

- 如何估算“计算时间是否足够覆盖 load”？
- CUTLASS pipeline 为什么 architecture-specific?

参考：R6, R13

题 68 | Hopper TMA 解决什么问题？

难度	★★★	频率	高	标签	TMA / Hopper

考察意图

考察从 thread-driven copy 到 hardware tensor transfer 的变化。

思路拆解

TMA（Tensor Memory Accelerator）可按 tensor descriptor 进行 1D-5D bulk transfer，在 global↔shared 之间异步搬运大块数据。少量线程发起后，硬件负责地址生成与数据移动，让更多 CUDA threads 专注计算。

标准回答

- 区别于 Ampere cp.async：TMA 的硬件地址生成与多维描述能力更强。
- 可与 async barriers 和 cluster 功能配合，构建端到端 pipeline。
- 单个 elected thread/少量线程可以驱动高带宽 transfer，减少每 lane address arithmetic。
- TMA 并不自动保证 coalescing/最佳 layout；descriptor 和 tile 仍需正确设计。

Gotchas：常见失分点

- 把 TMA 说成“Tensor Memory（TMEM）”。二者是不同概念。

面试官可能继续追问

- TMA 与 TMEM 区别?
- TMA 为什么适合 warp-specialized producer?

参考：R5, R13

题 69 | `cuda::pipeline` / `cuda::barrier` 在 Async Pipeline 中做什么？

难度	★★★	频率	中高	标签	Barrier / Pipeline

考察意图

考察异步操作正确性。

思路拆解

异步 load 发起后，consumer 必须知道“哪个 stage 已完成、何时可读、何时可覆盖”。pipeline/barrier primitives 用 phase/arrival/wait 等语义协调 producer 与 consumer。

标准回答

- producer 发起 async copy 并提交 stage。
- consumer 等待 stage ready 后计算。
- 消费完成后，该 buffer 才能被 producer 复用。
- Hopper TMA / async proxy 还涉及更细的 memory ordering/proxy fence 语义，高级岗位应知道不能只靠普通 load/store 顺序猜测。

Gotchas：常见失分点

- buffer 还没传完就读取。
- 忘记 reuse barrier，producer 覆盖仍在使用的数据。

面试官可能继续追问

- mbarrier 的 phase 概念是什么?

- async proxy fence 为什么存在?

参考：R13

题 70 | Kernel Fusion 为什么经常有效？什么时候会适得其反？

难度	★★☆	频率	极高	标签	Fusion / Memory

考察意图

考察 memory traffic 与资源压力的权衡。

思路拆解

Fusion 把多个 kernels 的中间结果留在 register/shared/cache 范围，减少 global store/load 与 launch overhead。对 AI inference 的 elementwise/epilogue 特别常见。

标准回答

- 主要收益：减少 HBM bytes、kernel launches、同步边界。
- 主要风险：code size、register live ranges、shared memory、divergence 增加，使 occupancy 或 instruction cache 变差。
- Fusion 还可能减少并行度：原本两个阶段可以有不同最佳 grid/block 配置。
- 是否融合要用 end-to-end profile 验证。

Gotchas：常见失分点

- “能融合就全部融合”。

面试官可能继续追问

- 为什么 FlashAttention 属于算法级 fusion?
- epilogue fusion 为什么通常收益高?

参考：R2, R4

本章复盘：面试前必须能说清的因果链

- 异步的价值来自 overlap，而不是 API 名字。
- Software pipeline 用额外 stage/buffer 换 latency hiding。
- Hopper TMA 将多维地址生成和大块数据搬运更多交给硬件。

第八章 Nsight、Roofline 与性能诊断

建立可证伪、可复现的性能工程方法。

本章抓手 Systems→Compute · Roofline/SOL · stall interpretation

题 71 | 优化 CUDA Kernel 的第一步是什么？

难度	频率	标签
★☆☆	极高	Profiling / 方法论

30 秒答案 先测量，再定位，再改；没有 baseline 的“优化”不可证伪。

考察意图

考察你是否具备性能工程纪律。

思路拆解

第一步是建立可信 baseline 和 correctness，而不是开始改 shared memory。优化循环应该是 Measure → Bottleneck → Hypothesis → Change → Measure。

标准回答

- 确认输出正确、输入代表真实 workload。
- 先用 Nsight Systems 看端到端时间线，找到真正 hot kernels / transfer / synchronization。
- 再用 Nsight Compute 深入 kernel，建立 bottleneck 假设。
- 每次只做可解释的改动，记录 before/after metrics。

Gotchas：常见失分点

- 凭经验直接重写，没有 baseline。

面试官可能继续追问

- 为什么先 Systems 再 Compute?
- 如何避免 profiler 本身的 measurement perturbation?

参考：R4, R10

题 72 | 如何判断 Memory-Bound 还是 Compute-Bound？

难度	频率	标签
★★☆	极高	Roofline / AI

考察意图

考察 Roofline 的核心。

思路拆解

计算 arithmetic intensity（AI = FLOPs / bytes from the limiting memory level），比较机器 compute peak / memory bandwidth 的 ridge point。AI 低通常受 bandwidth ceiling，AI 高才有机会进入 compute ceiling。

标准回答

- Roofline：P = min(P_compute_peak, BW × AI)。
- 注意 bytes 应针对你分析的 memory level；L2 与 HBM 可有不同 roof。
- 实际 kernel 还可能 latency-bound、launch-bound、dependency-bound，因此 Roofline 是第一层分类，不是全部诊断。

Gotchas：常见失分点

- 只看 SM utilization 与 DRAM utilization 二选一。

面试官可能继续追问

- 如何计算 GEMM AI?
- 为什么小 GEMM 可能既没到 compute roof 也没到 memory roof?

参考：R4

题 73 | Roofline Model 如何指导具体优化？

难度	★★☆	频率	极高	标签	Roofline / 方法论

考察意图

考察从模型到动作。

思路拆解

若点靠近 memory roof，减少 bytes/增加 reuse/提高 AI；若靠近 compute roof，优化 Tensor Core、指令效率、precision、pipeline。若远离两条 roof，说明还有 latency、occupancy、divergence、launch 等其他问题。

标准回答

- Memory-bound：coalescing、tiling、fusion、lower precision。
- Compute-bound：Tensor Core、MMA shape、instruction mix、数学近似/低精度。
- 低于两 roof：看 stalls、eligible warps、small grid、serial dependencies。
- 优化后重新在 Roofline 上观察点是否向目标 roof 移动。

Gotchas：常见失分点

- 把 Roofline 当成只有一条 DRAM roof 的图。

面试官可能继续追问

- 如何构建 L1/L2/DRAM hierarchical roofline?
- Tensor Core roof 与 CUDA Core roof 如何区分?

参考：R4

题 74 | Nsight Systems 与 Nsight Compute 怎么分工？

难度	★☆☆	频率	极高	标签	Nsight / 工具

考察意图

考察 profiling 工具链。

思路拆解

Systems 看“应用级时间线与 CPU-GPU orchestration”；Compute 看“单 kernel 的微架构效率”。

标准回答

- Nsight Systems：CPU threads、CUDA API、streams、memcpy、kernel gaps、NCCL、同步。
- Nsight Compute：SM/memory throughput、occupancy、warp stalls、cache、instruction mix、source/SASS。
- 先 Systems 找 hot spot，再 Compute 深挖，避免优化一个占比很小的 kernel。

Gotchas：常见失分点

- 用 Nsight Compute profile 全程序所有 kernel，得到巨大且难解释的数据。

面试官可能继续追问

- 如何限定 kernel regex / launch skip/count?
- 什么时候用 sampling 而不是 replay counters?

参考：R4, R10

题 75 | SM Throughput 95% 是否说明 Kernel 已经最优？

难度	★★☆	频率	高	标签	Metrics / Compute

考察意图

考察“utilization 不等于 useful work”。

思路拆解

SM 很忙可能是在执行低效 address arithmetic、conversion、冗余计算或严重 divergence 后的多条路径。必须把 throughput 与 achieved useful FLOPs、instruction mix、warp efficiency 和 roofline 一起看。

标准回答

- 高 SM utilization 说明某些执行资源忙，不说明算法 work efficiency。
- 如果实际 useful FLOPs 远低于理论可达，可能 instruction mix 不理想。
- Tensor Core workload 还需看是否真正使用高吞吐 MMA path。

Gotchas：常见失分点

- 看到 95% 就停止优化。

面试官可能继续追问

- 如何区分 CUDA-core busy 与 Tensor-core busy?
- branch divergence 会不会让 SM 仍然很忙?

参考：R4

题 76 | DRAM Throughput 95% 意味着什么？

难度	频率	标签
★★☆	高	Metrics / Memory

考察意图

考察 bandwidth saturation 后该做什么。

思路拆解

如果 DRAM throughput 已接近实测/理论可达上限，继续“提高带宽利用率”空间很小；应减少 bytes per output 或提高数据复用，让同样结果需要更少 HBM 流量。

标准回答

- 先验证 actual bytes 是否大于 logical bytes（过度传输）。
- 若访问已高效，考虑 fusion、lower precision、compression、recompute、cache/shared reuse。
- 如果 workload 主要命中 L2，DRAM 95% 可能只出现在部分阶段；要结合 timeline。

Gotchas：常见失分点

- 把 bandwidth saturation 当性能无解。

面试官可能继续追问

- 如何用算法变换提高 AI?
- 什么时候更低精度真正减少 HBM bytes?

参考：R4

题 77 | 常见 Warp Stall Reasons 应如何解释？

难度	频率	标签
★★★	极高	Stall / Nsight

考察意图

考察是否能用 scheduler 状态构建因果链，而不是背指标名字。

思路拆解

Warp stall 要和相关 subsystem 指标一起看。典型如 long scoreboard 常关联高延迟 memory dependency；barrier 关联同步等待；not selected 表示 warp eligible 但本周期未被 scheduler 选中，未必是坏事。

标准回答

- Long scoreboard：常见为 L1TEX/更长 latency 数据依赖等待。
- Short scoreboard：较短 latency dependency/某些 shared/特殊路径等待。
- Barrier：CTA/warp 同步导致等待，需看负载不均或 barrier 频率。
- Not selected：有足够 eligible warps 时调度竞争，可能意味着延迟隐藏是健康的。
- No eligible / issue slot unused 才更值得追问为什么没有可发射工作。

Gotchas：常见失分点

- 看到某个 stall 百分比最高就单独优化它。

面试官可能继续追问

- 如何区分 memory latency 与 memory bandwidth bound?
- barrier stall 高可能由什么数据不均衡导致?

参考：R4

题 78 | 为什么很短或 Block 很少的 Kernel，Occupancy 指标可能没意义？

难度	★★☆	频率	高	标签	Small Kernel / Tail

考察意图

考察 grid-level parallelism。

思路拆解

Occupancy 描述“若有足够 blocks，单 SM 可驻留多少 warp”，但如果整个 grid 只有少数 blocks，许多 SM 根本拿不到工作，瓶颈是 insufficient parallelism 或 launch overhead。

标准回答

- small grid 先看 active SM 数、waves、kernel duration。
- 通过 batch、fusion、persistent work queue、增加并行维度等扩大工作。
- 对于微秒级 kernel，CPU launch overhead 可能占主要比例，CUDA Graph 更重要。

Gotchas：常见失分点

- 对 4-block kernel 花大量时间调 theoretical occupancy。

面试官可能继续追问

- 什么是 tail effect?
- 为什么 batched small GEMM 需要专门 kernel?

参考：R4

题 79 | 什么是 Speed of Light（SOL）分析？

难度	★★☆	频率	中高	标签	SOL / Nsight

考察意图

考察把实际吞吐归一化到硬件峰值。

思路拆解

SOL 把某子系统实际吞吐与架构峰值比较，帮助判断 compute、memory、某 pipeline 哪个接近饱和。但“接近某 roof”仍需结合 useful work 和算法目标。

标准回答

- SM SOL 高、Memory SOL 低：更偏 compute/issue 侧。
- Memory SOL 高、SM SOL 低：更偏 bandwidth/transfer。
- 两者都低：可能 latency、small grid、dependency、divergence 或调度问题。
- 使用实际工作频率与 profiler 的 normalized metrics，避免只拿 marketing peak 生硬比较。

Gotchas：常见失分点

- 把 SOL 百分比当统一的“GPU utilization”。

面试官可能继续追问

- 为什么不同 pipeline 有不同 peak?
- 如何结合 Roofline 与 SOL?

参考：R4

题 80 | 如何证明一个 CUDA 优化“真的有效”？

难度	★★☆	频率	极高	标签	Benchmark / 工程

考察意图

考察科学实验式性能验证。

思路拆解

需要控制变量：同样输入、同样正确性/精度、warm-up、多次重复、稳定电源/频率条件；用 CUDA events 或 profiler 记录，并解释 metric 的因果变化。

标准回答

- 报告 median/p50/p95 或至少多次平均与方差，不依赖单次。
- 同时记录硬件、driver、CUDA、编译 flags、GPU clocks/power mode。
- 做 correctness regression，包括边界 shape。
- 最好给出“为什么变快”的证据，如 DRAM bytes -35%、bank conflicts 消失、Tensor Core utilization 上升。

Gotchas：常见失分点

- 只报告一次 2ms→1ms，没有测试方法。

面试官可能继续追问

- 如何做微基准避免 cache warm 状态误导?
- 编译器 dead-code elimination 怎么防?

参考：R2, R4

本章复盘：面试前必须能说清的因果链

- 先 Systems 找“时间去哪了”，再 Compute 解释 hot kernel 为什么慢。
- Roofline 给方向，stall/traffic/resource counters 给因果。
- 每次优化必须有 correctness + timing + counter evidence。

第九章 GEMM、Tensor Core 与 CUTLASS

理解现代 AI Kernel 的分层 tiling 与 warp specialization。

本章抓手 hierarchical tiling · Tensor Core · CUTLASS schedule

题 81 | 为什么 GEMM 是 CUDA 性能优化的“综合大题”？

难度	★★☆	频率	极高	标签	GEMM / 综合

考察意图

考察能否把 memory hierarchy 与 compute pipeline 串起来。

思路拆解

GEMM 同时要求 global coalescing、shared-memory tiling、register blocking、Tensor Core/MMA、software pipeline、occupancy/ILP、epilogue 和 scheduler。它几乎覆盖现代 CUDA kernel 的全部核心主题。

标准回答

- 数据路径：HBM/L2 → shared → registers/TMEM → Tensor Core。
- 并行层次：CTA tile → warp/warp-group tile → MMA tile → thread/register fragment。
- 控制层：async copy/TMA、barrier、multi-stage pipeline、warp specialization。
- 因此“会写 naive tiled GEMM”和“理解 CUTLASS 级 GEMM”是两个层次。

Gotchas：常见失分点

- 只会 shared-memory textbook GEMM，就认为掌握现代 GEMM。

面试官可能继续追问

- CUTLASS mainloop 与 epilogue 分别做什么?
- 为什么 attention 优化也大量借鉴 GEMM tiling?

参考：R6

题 82 | GEMM 为什么 Arithmetic Intensity 很高？

难度	★★☆	频率	高	标签	AI / GEMM

考察意图

考察 FLOPs/bytes 与数据复用。

思路拆解

矩阵乘法的计算量为 O(MNK)，通过 blocking 同一个 A/B tile 可以被多个输出元素反复使用，因此 bytes 增长可以慢于 FLOPs，问题在足够大时可走向 compute-bound。

标准回答

- 理想 GEMM 每个输入 tile 被多次 reuse。
- 小矩阵/skinny shape 的 reuse 不足或并行度不足，可能仍是 memory/latency-bound。
- 实际 AI 要考虑读 C、写 C、epilogue、cache hit 和 dtype bytes。

Gotchas：常见失分点

- 只用数学 O(MNK) 就断言任何 GEMM compute-bound。

面试官可能继续追问

- 如何估算一个 tile 的 reuse factor?
- 为什么 K 很小的 GEMM 常不饱和 Tensor Core?

参考：R4, R6

题 83 | GEMM 为什么需要多级 Tiling？

难度	★★★	频率	极高	标签	CUTLASS / Tiling

考察意图

考察如何映射 memory hierarchy。

思路拆解

一个 tile 尺寸无法同时匹配 HBM、shared、register 和 MMA 的不同容量/吞吐，因此现代 GEMM 分层：CTA tile 控制 global/shared reuse，warp tile 控制 shared→register reuse，MMA/thread tile 对齐计算指令。

标准回答

- CTA tile：由整个 thread block 负责，通常存入 shared。
- Warp/warp-group tile：一组 warps 处理输出子块。
- MMA tile：匹配 Tensor Core 指令形状。
- Thread/register tile：每线程/fragment 持有 accumulator，最大化 register reuse 与 ILP。

Gotchas：常见失分点

- 把 CUTLASS 层级只当软件抽象，不解释对应硬件资源。

面试官可能继续追问

- tile shape 如何影响 shared bank conflicts?
- 为什么 accumulator register 是主要压力?

参考：R6

题 84 | Tensor Core 与普通 CUDA Core 的区别是什么？

难度	★★☆	频率	极高	标签	Tensor Core / MMA

考察意图

考察 AI kernel 的基础计算单元。

思路拆解

CUDA cores 执行标量/向量风格算术指令；Tensor Cores 提供专用矩阵乘加吞吐，以特定 tile/layout/dtype 进行 MMA。要获得高性能，数据必须满足相应 shape、layout、alignment 和 pipeline。

标准回答

- 抽象操作为 D = A×B + C 的矩阵块。
- 支持的 dtype 随架构扩展，包括 TF32、FP16/BF16、FP8、FP4/block-scaled 等。
- Tensor Core 高峰值只有在足够工作量和有效 feed 时才有意义；memory/pipeline 不够仍会饿死 Tensor Core。

Gotchas：常见失分点

- “用了 half 就自动走 Tensor Core”。

面试官可能继续追问

- 如何确认 SASS/PTX 真的发出 MMA?
- TF32 在数值上与 FP32 有何差别?

参考：R6, R14

题 85 | FP32、TF32、FP16、BF16、FP8、FP4 怎么选？

难度	★★★	频率	极高	标签	Precision / AI

考察意图

考察性能、范围、精度与硬件吞吐的联合判断。

思路拆解

低精度通常同时减少 memory bytes 并提高 Tensor Core throughput，但可能带来范围/舍入误差，需要 scaling、accumulation dtype 与数值验证。选 dtype 是模型/算法约束，不是单纯性能选项。

标准回答

- FP32：通用精度；TF32 用于部分 FP32 matrix math，保留较大指数范围但尾数更少。
- BF16 指数范围接近 FP32，常用于训练；FP16 尾数/指数配置不同。
- FP8/FP4 对 scaling 和量化策略更敏感，在 Blackwell 等架构可提供更高 AI 吞吐。

- 很多 kernel 采用低精度 input + 更高精度 accumulation。

Gotchas：常见失分点

- 只比较 bit 数，不谈动态范围、accumulator 和 scaling。

面试官可能继续追问

- 为什么 BF16 训练常比 FP16 更稳?
- block scaling 是什么?

参考：R14, R6

题 86 | WMMA、PTX MMA、CUTLASS、cuBLAS 各处于什么抽象层？

难度	★★★	频率	高	标签	API / 抽象

考察意图

考察什么时候下沉到低层。

思路拆解

从高到低可理解为：cuBLAS/cuBLASLt 提供库级算法；CUTLASS/CuTe 提供可组合 kernel building blocks；WMMA 是较高层 warp matrix API；PTX MMA/WGMMA/TCGen 指令提供更直接控制。

标准回答

- 越低层控制越强、architecture coupling 越高、维护成本越大。
- 对标准 GEMM 优先库；定制 epilogue/layout/schedule 时 CUTLASS 常是合理平衡。
- 直接 PTX 适合极端定制或研究，但要承担 ISA/架构复杂度。

Gotchas：常见失分点

- 把 CUTLASS 当成只包了一层 cuBLAS。

面试官可能继续追问

- CuTe layout algebra 解决什么?
- 什么时候 inline PTX 才值得?

参考：R6, R14

题 87 | CUTLASS 的核心设计思想是什么？

难度	★★★	频率	极高	标签	CUTLASS / 工程

考察意图

考察现代 CUDA 模板化 kernel 设计。

思路拆解

CUTLASS 把 GEMM/conv 等操作拆成 layout、copy atom、MMA atom、tile shape、mainloop、epilogue、scheduler 等可组合部件，并为不同架构提供特化 schedule。

标准回答

- “分层 tiling + 可组合 primitive + architecture specialization”是主线。
- CuTe 用 layout/tensor 抽象描述数据坐标与线程映射。
- Hopper/Blackwell 的数据搬运与 warp specialization 差异很大，因此 schedule 往往 architecture-specific。
- 使用 CUTLASS 仍需要理解 profiler、alignment、shape 和 resource trade-off。

Gotchas：常见失分点

- 认为模板参数越复杂只是“C++ 炫技”。

面试官可能继续追问

- mainloop 为什么与 epilogue 分离?
- collective abstraction 有什么作用?

参考：R6, R15

题 88 | 什么是 Warp Specialization？

难度	★★★	频率	极高	标签	Warp Specialization / 真题

30 秒答案 Warp specialization = 在同一 CTA 中让不同 warp/warp-group 承担不同 pipeline 角色，例如 TMA producer 与 MMA consumer。

考察意图

这是公开 NVIDIA 面经可核验的现代 CUDA 题。考察 producer-consumer 的线程角色分工。

思路拆解

传统 CTA 中所有 warps 往往执行相似 load+compute 逻辑；warp specialization 把一部分 warp/warp-group 专门做 TMA/data movement/控制，另一部分专门做 MMA/compute，通过 barriers/pipeline 协调。

标准回答

- Producer warps：发起 TMA、管理 stages、调度数据。
- Consumer warps：等待 tile ready，执行 MMA/accumulation。
- 优势是角色化资源分配与更清晰的异步 overlap；代价是 barrier、负载平衡和角色资源不对称。
- CUTLASS 明确指出 Hopper 与 Blackwell 的具体 warp-specialization 结构不同，但 producer/consumer 思想延续。

Gotchas：常见失分点

- 把 warp specialization 当成“不同 warp 处理不同输出 tile”这么普通的空间划分。

面试官可能继续追问

- 为什么 producer warp 可以用更少 registers?
- Blackwell 为什么需要不同结构?

参考：R12, R6

题 89 | 什么是 Persistent Kernel？

难度	★★★	频率	高	标签	Persistent / Scheduler

考察意图

考察减少 launch/调度开销与动态负载均衡。

思路拆解

Persistent kernel 让一组 CTAs 长时间 resident，通过循环从 work queue 或 tile scheduler 获取多份工作，而不是“一 block 一 tile 然后退出”。

标准回答

- 可 amortize kernel prologue/launch/scheduling overhead。
- 可做动态 load balancing，对 irregular work 有利。
- resident CTAs 数常按 SM 数/资源有意控制。
- 代价是自定义 work scheduling、fairness、同步和更复杂的尾部处理。

Gotchas：常见失分点

- 把 grid-stride loop 和 persistent kernel 完全等同。

面试官可能继续追问

- CUTLASS persistent cooperative kernel 为什么有两个 consumer warp groups?
- persistent kernel 如何避免 work queue contention?

参考：R6

题 90 | 什么是 Split-K / Stream-K？什么时候能解决 GEMM 利用率问题？

难度	★★★	频率	高	标签	Stream-K / GEMM

考察意图

考察非标准形状下的并行度。

思路拆解

当 M×N tiles 数不足以填满 GPU，而 K 很大时，可以把 K 维也拆分给多个 workers，产生 partial sums，再归并。Split-K 通常静态拆 K；Stream-K 更强调把 K 维 work units 连续分配以改善 wave/tile imbalance。

标准回答

- 收益：增加并行 work units，改善小 M/N、大 K 的 GPU utilization。
- 代价：partial sums reduction、额外 memory/atomic/synchronization。
- 是否值得取决于 shape、dtype、epilogue 与 reduction 成本。

Gotchas：常见失分点

- 只说“把 K 分块”，不提 partial reduction。

面试官可能继续追问

- 为什么普通 data-parallel GEMM 不需要跨 block reduction?
- Stream-K 如何缓解 wave quantization?

参考：R6

本章复盘：面试前必须能说清的因果链

- 现代 GEMM 是 HBM→shared→register/TMEM→Tensor Core 的分层流水。
- Tiling、warp specialization、persistent schedule 都是为了提高 reuse 并隐藏数据搬运。
- CUTLASS 的价值是把这些机制做成可组合、架构特化的 building blocks。

第十章 Hopper、Blackwell 与 CUDA 13.x

Thread Block Cluster、DSM、TMA、TMEM、CUDA Tile 与工程化优化。

本章抓手 cluster/DSM · Blackwell TMEM · CUDA Graph/Allocator/Tile

题 91 | Hopper Thread Block Cluster 是什么？

难度	频率	标签
★★★	高	Hopper / Cluster

考察意图

考察 CC 9.0+ 的新层级。

思路拆解

Thread Block Cluster 在 Grid 与 Block 之间引入可编程协作层级。一组 blocks 被保证并发调度在相邻硬件范围内，可以做 cluster-level sync、访问 distributed shared memory，并协同驱动异步单元。

标准回答

- 层级扩展为 Grid → Cluster → Block → Thread。
- cluster 内 blocks 可使用 cluster group 同步。
- cluster size 影响 locality、可调度性与资源；不是越大越好。
- 普通 block 仍然只驻留一个 SM，但 cluster 让多个 blocks 跨 SM 协作。

Gotchas：常见失分点

- 说“一个 block 现在可以跨多个 SM”。

面试官可能继续追问

- `__cluster_dims__` 如何指定?
- cluster size 为什么影响 occupancy/scheduling?

参考：R5, R7

题 92 | 什么是 Distributed Shared Memory（DSM）？

难度	频率	标签
★★★	高	DSM / Hopper

考察意图

考察 cluster 内跨 block 数据共享。

思路拆解

DSM 把 cluster 中各 blocks 的 shared-memory segments 组成分布式地址空间，使 cluster 内线程可以对远端 block 的 shared memory 做 load/store/atomic，避免部分通信绕到 global memory。

标准回答

- 每 block 的 shared size 仍是独立配置，DSM 总容量约为 cluster blocks × per-block shared。
- 使用 remote DSM 前必须保证相关 blocks 已存在，并用 cluster sync 等正确管理 lifetime。
- 典型应用：大 histogram、跨 block reduction/aggregation、cluster data exchange。
- DSM 是低延迟中间层，但仍比本地 shared 更远，需谨慎布局。

Gotchas：常见失分点

- 认为 DSM 是所有 SM 全局共享的巨大 shared memory。

面试官可能继续追问

- 远端 DSM 与 local shared latency 是否相同?
- histogram bins 什么时候适合 DSM?

参考：R5, R3

题 93 | TMA 与 `cp.async` 的核心差别是什么？

难度	★★★	频率	极高	标签	TMA / cp.async

考察意图

考察 Ampere 与 Hopper 数据搬运范式。

思路拆解

`cp.async` 仍更接近 threads 参与的 global→shared 异步复制；TMA 使用 tensor descriptor 和硬件地址生成，可由少量线程发起大型、多维 bulk transfer，并支持更丰富的 tensor layout/cluster 场景。

标准回答

- 二者都服务于 copy/compute overlap。
- TMA 显著降低 per-thread address-generation 和 copy-loop overhead。
- TMA 与 async barriers、warp specialization 天然配合。
- 在简单小 copy 上，TMA 的 setup/descriptor 并不必然占优。

Gotchas：常见失分点

- 把 TMA 当成“更快的 memcpy”而不理解 programming model。

面试官可能继续追问

- 为什么 TMA 发起通常由 elected thread 完成?
- TMA descriptor 包含哪些逻辑信息?

参考：R5, R13

题 94 | 为什么 Hopper GEMM 适合 Producer-Consumer Warp Specialization？

难度	★★★	频率	高	标签	Hopper / GEMM

考察意图

考察 TMA 与 WGMMA/MMA 的并行喂数。

思路拆解

Hopper 可让 producer warp group 专门发起 TMA、推进 pipeline，consumer warp groups 专注 matrix math。这样 memory engine 与 Tensor Core 工作能更稳定重叠，也允许角色使用不同的 register budget。

标准回答

- producer 控制 descriptors、barriers、stage state，通常不需要大量 accumulators。
- consumer 需要较多 register 保存 accumulator/fragment。
- persistent CTA 与 warp specialization 结合，可以 amortize scheduler/prologue。
- 关键挑战是 producer 不饿、consumer 不饿以及 stage 生命周期正确。

Gotchas：常见失分点

- 只画 producer/consumer，不讨论 resource asymmetry。

面试官可能继续追问

- 为什么 `setmaxnreg`/register redistribution 在某些架构很重要?
- consumer 两组合作同 tile 有什么收益?

参考：R6, R5

题 95 | Blackwell Tensor Memory（TMEM）是什么？与 TMA 有什么区别？

难度	★★★	频率	中高	标签	Blackwell / TMEM

考察意图

考察 5th-gen Tensor Core 周边的新数据 locale。

思路拆解

TMEM 是 Blackwell/TCGen05 相关的专用 tensor memory 数据位置，用于 Tensor Core 数据/accumulator workflow；TMA 则是数据搬运加速器。二者名称接近但完全不同。

标准回答

- PTX TCGen05 提供 TMEM allocation/deallocation、TMEM↔register load/store、shared→TMEM 等相关指令。
- CUTLASS/CuTe 已把 `tmem` 暴露为 first-class data locale。

- TMEM 的价值在于匹配第五代 Tensor Core 的执行与数据路径，而不是通用 shared memory 替代品。
- 调试可使用针对 tensor memory access 的编译/检查工具。

Gotchas：常见失分点

- 把 Tensor Memory Accelerator（TMA）与 Tensor Memory（TMEM）混为一谈。

面试官可能继续追问

- tcgen05 为什么需要专门同步模型?
- TMEM 在 GEMM accumulator pipeline 中放什么?

参考：R14, R15

题 96 | 为什么 Blackwell 的 Warp Specialization 不能简单照搬 Hopper？

难度	★★★	频率	中高	标签	Blackwell / Schedule

考察意图

考察 architecture-aware tuning。

思路拆解

CUTLASS 官方明确指出 SM100/Blackwell 的 warp-specialization 结构与 Hopper 有显著不同。新 Tensor Core/TCGen05、TMEM、CTA pair/issue granularity 与数据移动能力改变了最佳角色划分和资源分配。

标准回答

- 同一算法在新架构上“能运行”不等于“接近最优”。
- kernel schedule 必须考虑新的 Tensor Core instructions、TMEM、TMA extensions 和 synchronization。
- 高层库/Tile abstraction 的意义之一就是吸收这类架构变化。

Gotchas：常见失分点

- 把 `-arch=sm_100` 重新编译当成完整迁移优化。

面试官可能继续追问

- 为什么性能可移植性与源码可移植性不同?
- CUTLASS schedule tag 如何体现 architecture specialization?

参考：R6, R15

题 97 | CUDA Graph 什么时候能明显提升性能？

难度	★★☆	频率	极高	标签	CUDA Graph / Launch

考察意图

考察 launch-bound workload。

思路拆解

当 workload 由大量短小、重复的 kernels/memcpy 构成，CPU launch 与 runtime orchestration 占比可能很高。Graph 把固定 DAG 捕获/实例化后高效 replay，减少每轮 launch 开销。

标准回答

- 特别适合 inference、iterative solver、固定/有限 shape 的重复执行。
- 收益取决于 kernel 本身持续时间；毫秒级单 kernel 通常不以 launch overhead 为主。
- Graph 需要处理参数更新、动态 shape、allocation 与 external dependency。
- Nsight Systems 可直观看到 graph 前后 kernel gaps 是否缩小。

Gotchas：常见失分点

- 把 CUDA Graph 当成让单个 kernel 算得更快。

面试官可能继续追问

- graph capture 有哪些限制?
- dynamic shapes 如何管理多个 graph exec?

参考：R1, R10

题 98 | 为什么 `cudaMallocAsync` / Stream-Ordered Allocator 更适合高性能程序？

难度	★★☆	频率	高	标签	Allocator / Runtime

考察意图

考察 allocation 与 stream ordering。

思路拆解

传统 `cudaMalloc/cudaFree` 成本高且容易引入全局性的同步/管理开销。`cudaMallocAsync/cudaFreeAsync` 基于 memory pool，在 stream order 中定义 allocation lifetime，便于重用 device memory。

标准回答

- allocation/free 成为 stream 中的有序操作，可与 dependency graph 结合。
- memory pool 避免频繁向驱动/OS 请求全新物理资源。
- 必须保证所有访问发生在 allocation 生效之后、free 生效之前；跨 stream 要显式建立依赖。
- 现代 Best Practices 明确推荐使用 stream-ordered pool allocator 管理设备内存。

Gotchas：常见失分点

- 把异步 allocator 当成无生命周期约束的普通 malloc。

面试官可能继续追问

- 跨 stream 使用同一 allocation 如何保证安全?
- memory pool release threshold 有什么意义?

参考：R11, R2

题 99 | CUDA Tile C++ / Tile Programming 为什么值得关注？

难度	★★★	频率	中高	标签	CUDA 13.3 / Tile

考察意图

考察 2026 年的新编程模型趋势。

思路拆解

CUDA 13.3 首次正式引入 CUDA Tile C++ API，把“tile”而非 thread 作为主要程序抽象之一，旨在更自然表达 tensor/tile 计算，并让编译器/IR 吸收更多底层线程与 Tensor Core 细节。

标准回答

- CUDA 13.3 中 Tile C++ 使用 `cuda_tile.h`、`cuda::tiles`，需要 C++20，启用 `--enable-tile`。
- `__tile_global__` 表示 tile kernel；Tile IR 将 GPU 建模为 tile-based processor，而 PTX 传统上以 SIMT 为中心。
- 目标之一是 performance portability：让硬件 Tensor Core/线程映射演进时，程序不必手工追所有低层细节。
- SIMT 与 Tile 可共存；高级岗位应比较 CUDA Tile、Triton、CuTe/CUTLASS 的抽象边界。

Gotchas：常见失分点

- 把 CUDA Tile 当成“CUDA thread block 的新名字”。

面试官可能继续追问

- Tile IR 与 PTX 有何根本抽象差别?
- 什么时候仍需要 SIMT/inline PTX?

参考：R16, R17, R18

题 100 | 给你一个比理论峰值慢 20 倍的 CUDA Kernel，你如何系统优化？

难度	★★★	频率	必问	标签	综合 / 性能工程

30 秒答案 正确性→Systems 定位→Compute/Roofline 分类→Memory/Execution/Resource 因果链→单变量优化→重新测量。

考察意图

这是整套题库的总题。面试官在看你有没有一套稳定、可证伪的性能诊断流程。

思路拆解

优秀回答应从 correctness 和端到端 profile 开始，而不是猜“肯定 memory-bound”。然后用 Roofline、memory traffic、warp stalls、resource/occupancy、instruction mix 一层层缩小瓶颈，做单变量实验，最后验证优化后的因果指标。

标准回答

- Step 1：建立 correctness/reference、代表性 input、稳定 benchmark。
- Step 2：Nsight Systems 找端到端 hot spot：launch、copy、sync、NCCL 还是 kernel。
- Step 3：Nsight Compute + Roofline 分类：bandwidth、compute、latency、small-grid/launch。
- Step 4：Memory：coalescing、actual/requested bytes、L2 hit、bank conflict、shared reuse。
- Step 5：Execution：eligible warps、stall reasons、divergence、instruction mix、Tensor Core usage。
- Step 6：Resources：register、spill、shared/block、active blocks/SM、occupancy 与 ILP。
- Step 7：按瓶颈应用 tiling/fusion/vectorization/async copy/TMA/software pipeline/precision/Tensor Core 等。
- Step 8：重新 profile；只有“时间变短 + correctness 保持 + 指标变化符合假设”才接受优化。

代码 / 白板骨架

```
Performance Engineering Loop

Correctness
   ↓
End-to-End Timeline (Nsight Systems)
   ↓
Hot Kernel
   ↓
Roofline + Nsight Compute
   ↓
Memory / Compute / Latency / Launch / Parallelism ?
   ↓
One hypothesis → One change
   ↓
Benchmark + Counter evidence
   ↓
Keep or Revert
```

Gotchas：常见失分点

- 第一句话就说“加 shared memory”。
- 一次改 5 件事，最后无法解释为什么变快。
- 只追 peak utilization，不看实际算法 bytes/FLOPs。

面试官可能继续追问

- 如果 DRAM 和 SM 都只有 30% SOL，你先查什么?
- 如果 kernel 已 memory roof 95%，下一步怎么提高吞吐?
- 如果 fusion 后慢了，你会检查哪些 resource metrics?

参考：R2, R4, R6, R5

本章复盘：面试前必须能说清的因果链

- Hopper 扩展了 cluster/DSM/TMA；Blackwell 又引入 TCGen05/TMEM 等新路径。
- TMA 是 accelerator，TMEM 是 memory locale，二者不能混。
- CUDA 13.3 Tile C++ 体现“从 thread-centric 到 tile-centric”的新抽象趋势。

附录 A | 100 题一页速查

第一章 CUDA 执行模型与 SIMT

题号	问题	面试核心
01	Thread、Warp、Block、Grid、SM 分别是什么?	编程层级是 Grid→Block→Thread；硬件调度核心是 SM→Warp，warp=32 threads。
02	CUDA 为什么称 SIMT，而不是传统 SIMD?	SIMD 的抽象是“一条显式向量指令操作多个 lane”；SIMT 的抽象是“多个逻辑线程被硬件以 warp 形式共同发射”。
03	为什么 Warp 是 CUDA 性能分析最重要的单位?	Global memory coalescing 是把一个 warp 的地址请求合并成尽量少的 memory transactions。
04	Kernel launch 后，Block 是如何调度到 SM 的?	Grid 中 blocks 可被动态分派给可用 SM，程序通常不应依赖某个 block 被放在哪个 SM。
05	为什么 block size 常从 128/256 开始，而不是随便选?	blockDim 最好是 warp size 的整数倍；例如 100 threads 会生成 4 个 warps，其中最后一个只有 4 个 active lanes。
06	什么是 Grid-Stride Loop? 为什么它是 CUDA 常用模式?	初始索引 i = blockIdx.x * blockDim.x + threadIdx.x。
07	`__syncthreads()` 到底同步什么?	同一个 thread block 中所有尚未退出且需要参与该 barrier 的线程必须到达同一个同步点。
08	CUDA 有哪些同步层级?	warp：`__syncwarp(mask)` 和 warp-level primitives。
09	为什么 kernel launch 通常是异步的? 如何正确计时?	同 stream 内操作保持顺序，但 host 可在 GPU 工作未完成时继续执行。
10	为什么 GPU 更偏 throughput，而 CPU 更偏 latency?	CPU 用复杂缓存、分支预测、乱序执行等机制优化少量线程的单线程 latency。

第二章 GPU Memory Hierarchy

题号	问题	面试核心
11	请完整描述 GPU Memory Hierarchy。	Register/Shared 在片上；Global/Local/Constant/Texture 主要驻留 device memory；L1/L2 是硬件 cache。
12	Register 为什么快? 为什么又不能无限使用?	register access 延迟低，适合 accumulator、循环变量和 thread tile。
13	什么情况下变量会 Spill 到 Local Memory? 如何发现?	用 `nvcc -Xptxas=-v` 查看 registers 与 spill load/store。
14	Shared Memory 为什么快? 它解决的三个核心问	显式 staging 可提高 data reuse 和 arithmetic

	题是什么?	intensity。
15	L1 Cache 与 Shared Memory 的区别是什么?	L1 优点是透明，代码简单；缺点是 replacement、命中率不可完全控制。
16	L2 Cache 为什么对多 Block / 多 SM 工作负载重要?	L2 对跨 SM 的重复读取、kernel 间 temporal locality 很重要。
17	Constant Memory 什么访问模式最快?	适合小型只读参数、查表常量、每 warp 一致的标量。
18	Texture / Read-Only Cache 什么时候有价值?	二维 spatial locality 或采样/插值类 workload 是典型适用场景。
19	`cudaMalloc` 与 Unified Memory / `cudaMallocManaged` 怎么选?	`cudaMalloc` + 显式 memcpy 给数据移动更强的可控性。
20	Pinned Host Memory 为什么能提高 H2D/D2H 吞吐?	Pinned memory 通常可提高 PCIe/NVLink host-device transfer throughput。

第三章 Global Memory 与 Coalescing

题号	问题	面试核心
21	什么叫 Global Memory Coalescing?	对 compute capability 6.0+ 的常见简化：warp 请求会被服务成覆盖其地址所需的若干 32-byte transactions。
22	32 个线程连续读 32 个 float，理想需要多少 transaction?	先算 useful bytes：32 × 4 = 128 B。
23	什么是 Misaligned Access？现代 GPU 上为什么仍要关注?	先判断一个 warp 的地址跨度与 segment 边界。
24	为什么 Strided Access 很危险?	stride=1 往往最容易 coalesce。
25	AoS 与 SoA 哪个更适合 CUDA?	SoA：字段分离，适合同一 warp 批量读取同一字段。
26	Vectorized Load/Store（如 float4）什么时候有收益?	先保证 pointer alignment 与数据长度边界。
27	为什么 Matrix Transpose 是经典 CUDA 优化题?	第一层优化：使用 32×32 或类似 tile，使读写都 coalesced。
28	Effective Bandwidth 怎么计算?	例如 N 个 float 从 x 读、向 y 写：逻辑流量约 8N bytes。
29	Requested Bandwidth 与 Actual DRAM Bandwidth 有什么区别?	requested/actual 比例低通常说明 transaction utilization 差。
30	一个 Memory-Bound Kernel 最常见的优化方向有哪些?	改善 coalescing 和 alignment，减少 transaction waste。

第四章 Shared Memory、 Warp 与 Divergence

题号	问题	面试核心
31	Shared Memory Bank 是什么?	如果 warp 的 32 个请求落在 32 个不同 banks，可高度并行服务。
32	什么是 Bank Conflict? Broadcast 为什么是例外?	冲突的本质是“same bank + different addresses”。
33	为什么 `tile[32][32]` 的列访问会出现 32-Way Bank Conflict?	stride=32 words 与 bank count=32 发生周期共振。
34	为什么 `tile[32][33]` 能消除典型 Transpose Bank Conflict?	padding 改变的是 shared-memory address mapping，不是 global layout。
35	什么是 Warp Divergence?	关键是“同一 warp 内”是否分歧，而不是是否存在 branch。
36	`if` 一定产生 Divergence 吗? 如何重构控制流?	分析 predicate 在 32 lanes 上的分布。
37	Predication 是什么? 为什么“消灭 if”不一定更快?	短、简单分支常适合 predication。
38	`__shfl_sync` 为什么可能比 Shared Memory 交换更快?	通信范围是 warp 内，因此不能替代跨 warp/block 通信。
39	`__syncwarp()` 什么时候必须用?	shuffle/vote 自带其定义的同步语义，但 shared-memory 手工通信通常需要显式同步。
40	如何写一个 Warp-Level Reduction?	32 个值归约到 1 个需要 log2(32)=5 轮。

第五章 Occupancy、 Register 与 Latency Hiding

题号	问题	面试核心
41	什么是 Occupancy?	Occupancy 只回答“SM 上有多少 active warps”，不是“这些 warps 做得是否高效”。
42	为什么 100% Occupancy 不一定最快?	方案 A：更多 warps，但每线程资源少。
43	哪些资源会限制 Resident Blocks / Occupancy?	thread/block × blocks ≤ max threads/SM。
44	什么是 Register Pressure? 如何权衡?	扩大 thread tile、unroll、fusion 都可能增加 register live ranges。
45	如何查看 Kernel 使用多少 Registers / Shared Memory / Spills?	`nvcc -Xptxas=-v` 查看 registers、stack、spill stores/loads 等。
46	`-maxrregcount` 是不是越小越好?	先确认当前 kernel 是否真的 latency/occupancy-bound。
47	`__launch_bounds__` 用来解决什么问题?	第一参数可防止编译出在目标 block size 下资源过量的 kernel，并帮助前向兼容。
48	ILP 与 TLP 有什么区别?	TLP 的代理指标包括 occupancy、eligible

		warps。
49	GPU 如何用 Warp Scheduling 隐藏 Memory Latency?	resident warp 多不等于 eligible warp 多；如果所有 warps 都等同一内存依赖，occupancy 再高也可能一起 stall。
50	Block Size 应该如何系统调优，而不是猜 256?	建立 baseline 和 correctness check。

第六章 经典 CUDA Kernel 现场编程

题号	问题	面试核心
51	现场手写一个高性能 Reduction：你会如何逐版优化?	Reduction 的优化主线：减少 global atomics、减少 barriers/shared traffic、提高每线程工作量，最后用 warp shuffle。
52	为什么最原始 Interleaved Reduction 很慢?	改成 sequential addressing：`if (tid < s) data[tid] += data[tid+s]`。
53	Reduction 最后一 Warp 为什么适合用 Shuffle?	5 次 `shfl_down` 即可把 32 lanes 归约。
54	手写 Parallel Scan / Prefix Sum：Hillis-Steele 与 Blelloch 如何比较?	Hillis-Steele 每轮 offset 加倍，轮数 O(log N)，但每轮很多元素做加法。
55	Parallel Histogram 如何优化 Atomic Contention?	先判断 bins 数、分布 skew、数据类型和 shared capacity。
56	现场写 Optimized Matrix Transpose。	用 32×8 threads 处理 32×32 tile，可让每线程循环处理 4 行，控制 block threads=256。
57	手写 Tiled Matrix Multiplication，并解释为什么快。	global bytes 被多个 FMA 摊薄，arithmetic intensity 上升。
58	为什么 GEMM Tile 越大不一定越快?	tile shape 要匹配 M/N/K 与硬件 Tensor Core/MMA shape。
59	如何实现 Stream Compaction?	flags[i] = predicate(x[i])。
60	Top-K / Sort / Selection 什么时候不应该自己写 CUDA?	CUB 对 scan/reduce/select/sort 等 primitives 很成熟。

第七章 Stream、异步拷贝与 Software Pipeline

题号	问题	面试核心
61	CUDA Stream 是什么？不同 Stream 真的能并发吗?	不同 stream 并发不是保证：如果一个 kernel 已占满所有 SM/resource，另一个可能没有空间。
62	`cudaMemcpyAsync` 为什么不一定真的异步?	Pinned + non-default stream 是常见前提。
63	如何设计 H2D - Kernel - D2H 的流水重叠?	chunk 太小：launch/copy latency 占比高；太大：并发阶段少且 buffer 占用大。
64	CUDA Event 与 `cudaDeviceSynchronize()` 有	GPU timing 通常用 events 记录同一 stream 的

	什么区别?	start/stop。
65	Ampere 的 Async Global→Shared Copy 解决什么问题?	核心收益一：更早发起下一 tile load，隐藏 memory latency。
66	什么是 Software Pipelining?	Software pipelining = 把不同迭代的 load/compute 重叠，用额外 buffer/stage 换 latency hiding。
67	什么是 Double Buffering / Multi-Stage Pipeline?	buffer 数决定能同时在途的阶段数。
68	Hopper TMA 解决什么问题?	区别于 Ampere cp.async：TMA 的硬件地址生成与多维描述能力更强。
69	`cuda::pipeline` / `cuda::barrier` 在 Async Pipeline 中做什么?	producer 发起 async copy 并提交 stage。
70	Kernel Fusion 为什么经常有效？什么时候会适得其反?	主要收益：减少 HBM bytes、kernel launches、同步边界。

第八章 Nsight、Roofline 与性能诊断

题号	问题	面试核心
71	优化 CUDA Kernel 的第一步是什么?	先测量，再定位，再改；没有 baseline 的“优化”不可证伪。
72	如何判断 Memory-Bound 还是 Compute-Bound?	Roofline：P = min(P_compute_peak, BW × AI)。
73	Roofline Model 如何指导具体优化?	Memory-bound：coalescing、tiling、fusion、lower precision。
74	Nsight Systems 与 Nsight Compute 怎么分工?	Nsight Systems：CPU threads、CUDA API、streams、memcpy、kernel gaps、NCCL、同步。
75	SM Throughput 95% 是否说明 Kernel 已经最优?	高 SM utilization 说明某些执行资源忙，不说明算法 work efficiency。
76	DRAM Throughput 95% 意味着什么?	先验证 actual bytes 是否大于 logical bytes（过度传输）。
77	常见 Warp Stall Reasons 应如何解释?	Long scoreboard：常见为 L1TEX/更长 latency 数据依赖等待。
78	为什么很短或 Block 很少的 Kernel，Occupancy 指标可能没意义?	small grid 先看 active SM 数、waves、kernel duration。
79	什么是 Speed of Light（SOL）分析?	SM SOL 高、Memory SOL 低：更偏 compute/issue 侧。
80	如何证明一个 CUDA 优化“真的有效”？	报告 median/p50/p95 或至少多次平均与方差，不依赖单次。

第九章 GEMM、Tensor Core 与 CUTLASS

题号	问题	面试核心
81	为什么 GEMM 是 CUDA 性能优化的“综合大题”？	数据路径：HBM/L2 → shared → registers/TMEM → Tensor Core。
82	GEMM 为什么 Arithmetic Intensity 很高?	理想 GEMM 每个输入 tile 被多次 reuse。
83	GEMM 为什么需要多级 Tiling?	CTA tile：由整个 thread block 负责，通常存入 shared。
84	Tensor Core 与普通 CUDA Core 的区别是什么?	抽象操作为 D = A×B + C 的矩阵块。
85	FP32、TF32、FP16、BF16、FP8、FP4 怎么选?	FP32：通用精度；TF32 用于部分 FP32 matrix math，保留较大指数范围但尾数更少。
86	WMMA、PTX MMA、CUTLASS、cuBLAS 各处于什么抽象层?	越低层控制越强、architecture coupling 越高、维护成本越大。
87	CUTLASS 的核心设计思想是什么?	“分层 tiling + 可组合 primitive + architecture specialization”是主线。
88	什么是 Warp Specialization?	Warp specialization = 在同一 CTA 中让不同 warp/warp-group 承担不同 pipeline 角色，例如 TMA producer 与 MMA consumer。
89	什么是 Persistent Kernel?	可 amortize kernel prologue/launch/scheduling overhead。
90	什么是 Split-K / Stream-K？什么时候能解决 GEMM 利用率问题?	收益：增加并行 work units，改善小 M/N、大 K 的 GPU utilization。

第十章 Hopper、Blackwell 与 CUDA 13.x

题号	问题	面试核心
91	Hopper Thread Block Cluster 是什么?	层级扩展为 Grid → Cluster → Block → Thread。
92	什么是 Distributed Shared Memory（DSM）?	每 block 的 shared size 仍是独立配置，DSM 总容量约为 cluster blocks × per-block shared。
93	TMA 与 `cp.async` 的核心差别是什么?	二者都服务于 copy/compute overlap。
94	为什么 Hopper GEMM 适合 Producer-Consumer Warp Specialization?	producer 控制 descriptors、barriers、stage state，通常不需要大量 accumulators。
95	Blackwell Tensor Memory（TMEM）是什么？与 TMA 有什么区别?	PTX TCGen05 提供 TMEM allocation/deallocation、TMEM↔register load/store、shared→TMEM 等相关指令。
96	为什么 Blackwell 的 Warp Specialization 不能简单照搬 Hopper?	同一算法在新架构上“能运行”不等于“接近最优”。

97	CUDA Graph 什么时候能明显提升性能?	特别适合 inference、iterative solver、固定/有限 shape 的重复执行。
98	为什么 `cudaMallocAsync` / Stream-Ordered Allocator 更适合高性能程序?	allocation/free 成为 stream 中的有序操作，可与 dependency graph 结合。
99	CUDA Tile C++ / Tile Programming 为什么值得关注?	CUDA 13.3 中 Tile C++ 使用 `cuda_tile.h`、`cuda::tiles`，需要 C++20，启用 `--enable-tile`。
100	给你一个比理论峰值慢 20 倍的 CUDA Kernel，你如何系统优化?	正确性→Systems 定位→Compute/Roofline 分类→Memory/Execution/Resource 因果链→单变量优化→重新测量。

附录 B | 面试答题模板：30 秒 / 2 分钟 / 深挖

30 秒：先给结论

模板 “结论是 X。核心原因有三点：A 决定了硬件事务，B 影响资源/并行度，C 决定最终是否能隐藏延迟。实际我会用 profiler 的 Y/Z 指标验证。”

2 分钟：给因果链

1. 定义问题与作用域：warp/block/device?
2. 建立资源/数据流模型：数据在哪一层、一次搬多少 bytes、需要多少同步?
3. 指出主要 trade-off：memory vs compute、register vs occupancy、reuse vs shared capacity。
4. 给出优化动作，并说明为什么它针对这个瓶颈。
5. 说明 profiler 如何验证。

深挖：现场被追问时

- 主动给一个反例：说明某个“常见结论”在哪些条件下不成立。
- 给一段最小代码或地址映射，避免停留在口号。
- 把架构代际差异说清：Ampere cp.async、Hopper TMA/cluster、Blackwell TMEM/TCGen05。
- 如果不确定某个微架构常数，不猜数字；说明应查 Programming Guide/compute capability table，再给不依赖常数的分析方法。

附录 C | 参考资料与题源说明

本书正文为原创整理。技术事实优先依据 NVIDIA 官方 CUDA / Nsight / CUTLASS / PTX 文档；公开面经仅用于核验“题型确实出现过”，不把匿名面经答案当技术权威。

R1 NVIDIA CUDA Programming Guide, CUDA 13.3 documentation

https://docs.nvidia.com/cuda/cuda-programming-guide/

R2 NVIDIA CUDA C++ Best Practices Guide 13.3

https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/

R3 CUDA Programming Guide - Distributed Shared Memory / Thread Block Clusters

https://docs.nvidia.com/cuda/cuda-programming-guide/

R4 NVIDIA Nsight Compute - Kernel Profiling Guide

https://docs.nvidia.com/nsight-compute/

R5 NVIDIA Hopper Architecture In-Depth

https://developer.nvidia.com/blog/nvidia-hopper-architecture-in-depth/

R6 NVIDIA CUTLASS Documentation - Efficient GEMM

https://docs.nvidia.com/cutlass/latest/media/docs/cpp/efficient_gemm.html

R7 CUDA Programming Guide - C/C++ Language Extensions, Launch Bounds, Synchronization, Clusters

https://docs.nvidia.com/cuda/cuda-programming-guide/05-appendices/cpp-language-extensions.html

R8 NVIDIA Developer Blog - Using CUDA Warp-Level Primitives

https://developer.nvidia.com/blog/using-cuda-warp-level-primitives/

R9 CUDA C++ Memory Model

https://docs.nvidia.com/cuda/cuda-programming-guide/05-appendices/cuda-cpp-memory-model.html

R10 CUDA Runtime API - Streams / Events

https://docs.nvidia.com/cuda/cuda-runtime-api/

R11 CUDA Runtime API - Stream Ordered Memory Allocator

https://docs.nvidia.com/cuda/cuda-runtime-api/group__CUDART__MEMORY__POOLS.html

R12 Public NVIDIA interview reports: synchronization/reduction/memory hierarchy; software pipelining/warp specialization

https://www.glassdoor.com/Interview/cuda-interview-questions-SRCH_KO0%2C4.htm

R13 CUDA Programming Guide - Advanced Kernel Programming / Async Operations and TMA

https://docs.nvidia.com/cuda/cuda-programming-guide/03-advanced/advanced-kernel-programming.html

R14 NVIDIA PTX ISA 9.3 - TCGen05 and Tensor Memory

https://docs.nvidia.com/cuda/parallel-thread-execution/

R15 NVIDIA CUTLASS Changelog - Blackwell SM100 / TMEM / TCGen05

https://docs.nvidia.com/cutlass/latest/CHANGELOG.html

R16 CUDA Tile C++ API Reference 13.3 - General Principles

https://docs.nvidia.com/cuda/cuda-tile-cpp-api-reference/general_principles.html

R17 NVIDIA NVCC 13.3 - Tile Compilation in CUDA

https://docs.nvidia.com/cuda/cuda-compiler-driver-nvcc/tile-compilation-in-cuda.html

R18 NVIDIA Tile IR 13.3 - Introduction

https://docs.nvidia.com/cuda/tile-ir/13.3/sections/introduction.html

公开面经可核验题型

- CUDA DevTech 公开面经出现：CUDA synchronization、reduction 实现、memory hierarchy（对应本书题 7/8、51-53、11）。
- 公开 NVIDIA 面经出现：software pipelining、warp specialization（对应本书题 66、88）。
- 其余题目按 NVIDIA 官方性能指南、现代 CUDA/CUTLASS 能力与 GPU 岗常见深挖方式构造成“高频真题型”；不声称逐字来自某一家公司。

版本说明

技术口径更新至 2026-09：CUDA 13.3 文档、CUDA Tile C++ 初始正式版本、当前 CUTLASS/Hopper/Blackwell 文档。GPU 微架构细节会随 Toolkit/driver/architecture 演进，面试时应优先解释稳定的性能原理，再补充具体架构能力。

最后一页：你是否真正准备好了？ 能否在没有 PPT 的情况下，从 warp 地址模式解释 coalescing；从 bank mapping 解释 padding；从 resource equation 解释 occupancy；从 Roofline + stall counters 解释瓶颈；从 HBM→shared→register/TMEM→Tensor Core 解释现代 GEMM。若这五条都能做到，已经跨过“只会写 CUDA”到“会做 GPU 性能工程”的门槛。